

MOTO RACING MANAGER — BÍBLIA MESTRA DE CONSTRUÇÃO DO JOGO

Documento-mãe para desenvolvimento no Codex

STATUS DO DOCUMENTO

Versão inicial — Fundação, arquitetura, regras-mãe e contrato de desenvolvimento em 3 partes.

1. VISÃO ABSOLUTA DO PRODUTO

Este projeto não deve ser tratado como “mais um manager”. A meta é construir o simulador de gestão de equipe de motovelocidade mais profundo, convincente, elegante e recompensador possível dentro da arquitetura web planejada. O jogador assume o papel de fundador, proprietário e principal executivo de uma equipe de corrida de motos e deve viver a operação completa de uma organização profissional: criação da marca, escolha da base, contratação de pilotos e staff, negociação com fabricantes, fornecedores e patrocinadores, desenvolvimento técnico, gestão financeira, preparação de cada etapa, tomada de decisões durante a corrida, expansão da estrutura e construção de um legado esportivo.

A experiência deve transmitir três sensações simultâneas:

- 1) “Eu sou dono desta equipe.”
- 2) “Minhas decisões realmente mudam o desempenho.”
- 3) “O campeonato continua vivo mesmo quando eu não estou diretamente envolvido.”

O jogo deve combinar profundidade de manager, leitura esportiva clara, apresentação cinematográfica e acessibilidade. Complexidade nunca pode significar confusão.

2. FANTASIA CENTRAL DO JOGADOR

O jogador começa com recursos limitados e um sonho. Ele escolhe identidade, país, sede, nome, escudo, cores, filosofia esportiva e posicionamento de mercado. A partir daí monta uma estrutura realista: diretoria, engenharia, mecânicos, performance, análise de dados, marketing, comercial, jurídico, logística, saúde e preparação física.

O objetivo não é apenas ganhar corridas. O jogador deve construir uma instituição. Uma equipe pequena pode sobreviver por anos antes de conquistar um pódio; uma equipe rica pode fracassar por má gestão; um piloto desconhecido pode virar campeão; um patrocinador pode transformar o orçamento; um erro contratual pode comprometer temporadas; uma decisão estratégica sob chuva pode criar um momento histórico.

3. REGRA-MÃE DE DESIGN

Toda funcionalidade precisa responder a pelo menos uma destas perguntas:

- Isso aumenta a sensação de ser dono de uma equipe real?
- Isso cria decisões significativas?
- Isso gera histórias emergentes?
- Isso melhora a leitura da corrida e da temporada?
- Isso cria consequências de curto, médio ou longo prazo?

Se a resposta for “não” para todas, a funcionalidade deve ser removida ou redesenhada.

4. PILARES INVIOLÁVEIS

Pilar A — Realismo administrativo: contratos, orçamento, salários, bônus, logística, fornecedores, desenvolvimento, patrocinadores e staff precisam formar um ecossistema coerente.

Pilar B — Realismo esportivo: desempenho deve nascer de uma combinação de moto, piloto, equipe, preparação, pneus, acerto, clima, desgaste, estratégia, confiança e incidentes.

Pilar C — Mundo vivo: IA controla equipes rivais com objetivos, orçamento, reputação, contratações e evolução próprios.

Pilar D — Corrida legível e emocionante: ícones 2D das motos se movimentam em circuitos reconhecíveis, com posições, intervalos, setores, incidentes, estratégia e ultrapassagens visualmente claras.

Pilar E — Profundidade sem microgestão inútil: o jogador pode delegar áreas, mas deve conseguir assumir controle fino.

Pilar F — Apresentação premium: UI limpa, moderna, cinematográfica, com sensação de transmissão esportiva de alto nível.

Pilar G — Rejogabilidade: nenhuma carreira deve ser igual à anterior.

Pilar H — Consequência: decisões devem ter efeitos mensuráveis e persistentes.

5. ESCOPO TÉCNICO INICIAL

A primeira versão será criada para hospedagem no GitHub, priorizando HTML, CSS e JavaScript modular. O projeto deve nascer preparado para evolução, sem ser um único arquivo monolítico.

Estrutura sugerida:

- index.html
- /src
 - /core
 - /state
 - /data
 - /systems
 - /simulation
 - /race
 - /ui
 - /services
 - /utils
- /assets

- /ui
- /backgrounds
- /circuits
- /teams
- /riders
- /sponsors
- /brands
- /audio
- /data
 - championships
 - circuits
 - manufacturers
 - suppliers
 - sponsors
 - staff
 - riders
 - economy
 - balance
- /docs
- /tests

O código deve privilegiar separação entre dados, lógica e interface. Nomes, atributos e valores de balanceamento não devem ficar espalhados pelo código.

6. REGRA DE DADOS E CONTEÚDO REAL

O jogo deve estar preparado para utilizar fabricantes, fornecedores, patrocinadores, pilotos, circuitos e competições reais quando houver autorização/licença adequada. Todo conteúdo real deve ficar em camada de dados substituível, nunca enterrado na lógica do jogo.

A engine deve funcionar também com dados fictícios, permitindo:

- desenvolvimento sem depender de licença;
- troca rápida entre conteúdo real e fictício;
- pacotes de expansão;
- temporadas históricas ou alternativas;
- mods futuros.

Marcas e identidades visuais reais só devem ser distribuídas quando o projeto tiver os direitos necessários.

7. ESTRUTURA DE CAMPEONATO

O jogo deve suportar múltiplas categorias e campeonatos, com diferenças de orçamento, potência, prestígio, regras, quantidade de etapas, fornecedores e nível dos pilotos.

Cada campeonato deve possuir:

- identificação;
- reputação;
- regulamento;
- calendário;

- sistema de pontuação;
- limite ou liberdade de testes;
- quantidade de motos por equipe;
- regras de pneus;
- regras de motores;
- restrições de desenvolvimento;
- critérios de inscrição;
- premiação;
- objetivos comerciais;
- nível de exposição na mídia.

A arquitetura deve permitir que campeonatos sejam adicionados por JSON ou módulos de dados, sem refazer a engine.

8. CRIAÇÃO DA EQUIPE

No início de uma carreira, o usuário escolhe:

- nome da equipe;
- nome curto;
- país;
- cidade-base;
- cores;
- identidade visual;
- filosofia;
- categoria inicial;
- estrutura financeira;
- expectativa de investidores;
- nível de risco;
- fabricante ou modelo de fornecimento disponível;
- quantidade de pilotos.

Perfis de filosofia:

- Equipe formadora;
- Equipe tecnológica;
- Equipe agressiva comercialmente;
- Equipe de baixo custo;
- Equipe de estrelas;
- Projeto de fabricante;
- Independente premium.

A filosofia não é cosmética: altera bônus, expectativas, custos, facilidade de contratação e objetivos.

9. PROPRIETÁRIO E DIRETORIA

O jogador é proprietário e pode acumular ou delegar funções. Deve existir uma tela de diretoria com:

- visão estratégica;
- metas da temporada;
- orçamento;

- confiança de investidores;
- reputação;
- sustentabilidade financeira;
- risco operacional;
- clima interno;
- imagem pública;
- objetivos de 1, 3 e 5 anos.

Eventos de diretoria podem exigir escolhas:

- investir em infraestrutura ou desempenho imediato;
- contratar estrela ou desenvolver jovem;
- aceitar patrocinador controverso;
- vender participação;
- assumir dívida;
- trocar fabricante;
- encerrar contrato caro;
- priorizar resultado esportivo ou rentabilidade.

10. STAFF — ESTRUTURA HUMANA

Cada funcionário deve ter função, atributos, personalidade, salário, contrato, moral, reputação, experiência, especialidades e potencial.

Áreas mínimas:

- Team Principal / Diretor Executivo;
- Diretor Técnico;
- Engenheiro-chefe;
- Engenheiro de performance;
- Engenheiro de dados;
- Crew Chief;
- Mecânicos;
- Especialista em eletrônica;
- Especialista em suspensão;
- Especialista em pneus;
- Especialista em aerodinâmica quando aplicável;
- Estrategista;
- Analista de adversários;
- Diretor esportivo;
- Chefe de logística;
- Médico;
- Fisioterapeuta;
- Preparador físico;
- Psicólogo esportivo;
- Diretor comercial;
- Marketing;
- Social media;
- Relações públicas;
- Jurídico;
- Financeiro;

- Scouting.

Atributos não devem ser apenas uma nota geral. Exemplo de engenheiro: análise de dados, acerto, feedback, inovação, trabalho sob pressão, comunicação, consistência e conhecimento de categoria.

11. MERCADO DE STAFF

O mercado precisa simular oferta e demanda. Profissionais valorizam salário, duração do contrato, bônus, reputação da equipe, chance de vitória, estabilidade, localização, fabricante e papel.

Negociação deve envolver:

- salário-base;
- bônus por resultado;
- cláusulas;
- duração;
- multa de rescisão;
- função;
- promessa de infraestrutura;
- metas;
- início do contrato.

A IA também disputa profissionais.

12. PILOTOS

Pilotos precisam ser personagens esportivos complexos. Atributos mínimos:

- velocidade pura;
- classificação;
- largada;
- ritmo de corrida;
- ultrapassagem;
- defesa;
- chuva;
- gestão de pneus;
- consistência;
- agressividade;
- controle emocional;
- feedback técnico;
- adaptação;
- preparo físico;
- risco de erro;
- marketing;
- popularidade;
- trabalho em equipe;
- experiência;
- potencial.

Forma atual e atributos permanentes devem ser separados. Um piloto pode ser excelente, mas estar em fase ruim.

13. PERSONALIDADE E RELACIONAMENTOS

Pilotos e staff possuem traços de personalidade: ambicioso, leal, midiático, reservado, agressivo, técnico, impaciente, mentor, individualista, disciplinado.

Relacionamentos internos influenciam:

- confiança;
- compartilhamento de dados;
- moral;
- velocidade de desenvolvimento;
- risco de conflito;
- renovação;
- desempenho sob pressão.

Deve existir histórico de acontecimentos para justificar mudanças de relação.

14. SCOUTING E FORMAÇÃO

O jogo deve possuir mercado de jovens e categorias de base. Scouting não pode revelar tudo imediatamente. Informação depende da qualidade da rede de observação.

Cada relatório pode conter:

- nível estimado;
- potencial estimado;
- pontos fortes;
- pontos fracos;
- estilo;
- personalidade provável;
- custo;
- risco;
- nível de confiança do relatório.

15. FABRICANTES E FORNECEDORES

A equipe negocia com fabricantes e fornecedores de componentes. Cada parceiro deve ter:

- custo;
- performance;
- confiabilidade;
- suporte técnico;
- prioridade dada à equipe;
- prazo de entrega;
- capacidade de atualização;
- reputação;
- compatibilidade com estilo da equipe;
- exigências comerciais.

Tipos de fornecimento:

- moto completa;

- motor;
- eletrônica;
- suspensão;
- freios;
- pneus quando o regulamento permitir;
- combustível/lubrificantes;
- equipamentos;
- ferramentas;
- tecnologia de dados.

Contratos podem envolver exclusividade, bônus, metas, prioridade de peças e acesso antecipado a atualizações.

16. A MOTO COMO SISTEMA

A moto não deve ser representada por um único número. Deve possuir subsistemas:

- motor;
- entrega de potência;
- eletrônica;
- chassi;
- rigidez;
- geometria;
- suspensão dianteira;
- suspensão traseira;
- freios;
- aerodinâmica;
- refrigeração;
- transmissão;
- confiabilidade;
- consumo;
- desgaste;
- facilidade de acerto;
- comportamento com pneus;
- estabilidade;
- agilidade;
- tração;
- velocidade final;
- aceleração.

Cada pista valoriza atributos diferentes.

17. DESENVOLVIMENTO TÉCNICO

P&D precisa funcionar por projetos e não por simples botão de “+1”.

Cada projeto possui:

- objetivo;
- custo;
- prazo;
- chance de sucesso;

- risco técnico;
- pessoal alocado;
- dependências;
- ganho previsto;
- ganho real após testes;
- efeito colateral possível.

Exemplos: novo pacote aerodinâmico, atualização eletrônica, geometria revisada, melhoria de refrigeração, redução de peso, novo mapa de motor, peça mais confiável.

18. INFRAESTRUTURA

A sede da equipe pode crescer em módulos:

- oficina;
- bancada de motor;
- laboratório de eletrônica;
- simulador;
- centro de dados;
- área de composição e engenharia;
- centro de performance humana;
- fisioterapia;
- departamento comercial;
- estúdio de mídia;
- armazém;
- logística;
- hospitality.

Cada instalação altera capacidade, custo fixo, produtividade e atratividade de profissionais.

19. FINANÇAS

O sistema financeiro deve ser central e transparente.

Receitas:

- patrocinadores;
- premiação;
- investimento;
- merchandising;
- hospitalidade;
- acordos de fabricante;
- licenciamento;
- bônus;
- venda de ativos quando aplicável.

Despesas:

- salários;
- bônus;
- peças;
- desenvolvimento;
- inscrição;

- viagens;
- logística;
- hospitalidade;
- manutenção;
- infraestrutura;
- marketing;
- seguros;
- multas;
- rescisões;
- contratação;
- operações.

O jogo deve diferenciar caixa, orçamento anual, compromissos futuros e projeção.

20. PATROCÍNIOS E COMERCIAL

Patrocinadores possuem setor, reputação, orçamento, mercados-alvo, tolerância a risco, exigências e objetivos.

A negociação deve permitir:

- valor fixo;
- bônus por pódio/vitória/título;
- duração;
- exclusividade;
- espaço de marca;
- ativações;
- presença digital;
- eventos;
- cláusulas de reputação.

O jogador deve poder montar um portfólio equilibrado. Um patrocinador principal pode exigir cores, exposição ou piloto popular.

21. MARKETING, MÍDIA E FÃS

A equipe possui marca e comunidade. Métricas:

- base de fãs;
- engajamento;
- alcance;
- valor de mídia;
- reputação;
- afinidade por país;
- venda de produtos;
- interesse de patrocinadores.

Ações:

- campanhas;
- documentários internos;
- conteúdos de bastidores;
- eventos;

- lançamentos de pintura;
- entrevistas;
- ações sociais;
- ativações com patrocinador.

Crises de comunicação podem ocorrer e exigem respostas.

22. CALENDÁRIO E LOGÍSTICA

Cada etapa deve ter custos e desafios logísticos. A distância entre sede e circuito, transporte, alfândega, prazos e risco de atraso devem impactar planejamento.

Fluxo de semana de corrida:

- 1) preparação e briefing;
- 2) definição de objetivos;
- 3) transporte e disponibilidade de peças;
- 4) treinos;
- 5) análise;
- 6) classificação;
- 7) corrida;
- 8) pós-corrida;
- 9) recuperação;
- 10) retorno e desenvolvimento.

23. CIRCUITOS

Os circuitos precisam ser entidades técnicas, não apenas imagens.

Atributos:

- comprimento;
- número de curvas;
- sentido;
- altitude;
- abrasividade;
- exigência de freio;
- exigência de tração;
- exigência aerodinâmica;
- velocidade média;
- oportunidades de ultrapassagem;
- risco de chuva;
- temperatura típica;
- desgaste;
- dificuldade;
- importância histórica;
- popularidade.

A representação 2D deve respeitar o formato do traçado e permitir leitura intuitiva de posição.

24. SISTEMA DE CORRIDA 2D — PRINCÍPIO

A corrida será exibida em circuito 2D com ícones de motos se movendo pela pista. O visual deve parecer uma central de comando esportiva moderna e não um simples mapa animado.

O motor de simulação deve rodar separado da renderização. A interface lê o estado da simulação e anima os ícones.

Cada competidor possui posição contínua de pista, volta, setor, ritmo, estado de pneu, combustível quando aplicável, desgaste, risco, estratégia e incidentes.

25. MODELO DE TEMPO DE VOLTA

O tempo de volta deve resultar de múltiplos componentes:

base do circuito

- + desempenho da moto para o perfil da pista
- + habilidade do piloto
- + forma
- + confiança
- + acerto
- + pneu
- + desgaste
- + temperatura
- + tráfego
- + clima
- + fadiga
- + dano
- + aleatoriedade controlada

Nunca usar aleatoriedade pura para decidir vencedor. A aleatoriedade apenas cria variação dentro de uma faixa coerente.

26. SETORES E MICRO-SIMULAÇÃO

A corrida deve ser calculada por setores ou microsegmentos. Cada trecho possui características: reta, curva lenta, curva média, curva rápida, frenagem, aceleração, mudança de direção.

Isso permite:

- motos com perfis diferentes ganharem tempo em pontos distintos;
- ultrapassagens contextualizadas;
- erros em locais plausíveis;
- desgaste coerente;
- visual de gaps mais realista.

27. ULTRAPASSAGENS

Uma tentativa de ultrapassagem considera:

- diferença de ritmo;
- posição na pista;
- trecho;
- velocidade final;
- frenagem;

- agressividade;
- habilidade ofensiva;
- habilidade defensiva;
- pneu;
- confiança;
- risco;
- tráfego.

Resultados possíveis:

- ultrapassagem limpa;
- defesa bem-sucedida;
- perda de tempo;
- contato;
- erro;
- saída de pista;
- queda.

O jogador deve entender por que algo ocorreu por telemetria simplificada, feed de eventos e contexto.

28. INCIDENTES E QUEDAS

Incidentes precisam ser raros o suficiente para não parecer loteria e frequentes o suficiente para manter tensão.

Probabilidade influenciada por:

- chuva;
- pista;
- pressão;
- agressividade;
- fadiga;
- dano;
- pneu;
- histórico;
- tráfego;
- confiabilidade;
- erros prévios.

A consequência pode ser perda de tempo, dano parcial, abandono, lesão e impacto de reputação.

29. CLIMA

Clima deve evoluir durante sessões. Variáveis:

- temperatura do ar;
- temperatura da pista;
- chance de chuva;
- intensidade;
- pista molhada;
- secagem;

- vento.

A previsão tem margem de erro conforme a qualidade da equipe.

30. PNEUS E ESTRATÉGIA

Pneus possuem compostos, janela ideal, aquecimento, degradação, aderência no seco/molhado e compatibilidade com piloto/moto.

O jogador define:

- escolha;
- ritmo-alvo;
- agressividade;
- preservação;
- resposta a mudanças climáticas;
- plano alternativo.

Estratégia deve ser contextual, não uma tela separada da simulação.

31. ACERTO

O acerto deve permitir profundidade gradual. Modo básico mostra efeitos em linguagem natural; modo avançado expõe parâmetros.

Áreas:

- geometria;
- suspensão;
- freio-motor;
- eletrônica;
- controle de tração;
- entrega de potência;
- relação;
- downforce quando aplicável.

Feedback de piloto e engenheiro ajuda a encontrar a janela ideal.

32. TELEMETRIA

A interface deve oferecer:

- tempos;
- setores;
- gaps;
- tendência;
- velocidade;
- consistência;
- degradação;
- temperatura;
- alertas;
- comparação entre pilotos.

O jogador não precisa ser engenheiro real para usar a informação. Cada dado deve gerar interpretação útil.

33. IA DAS EQUIPES ADVERSÁRIAS

Cada equipe IA possui:

- orçamento;
- filosofia;
- agressividade de mercado;
- tolerância a risco;
- qualidade administrativa;
- metas;
- reputação;
- relacionamento com fabricante;
- prioridades.

A IA deve contratar, demitir, renovar, desenvolver, negociar, escolher estratégia e reagir à temporada.

Ela não pode “trapacear” vendo informações privadas do jogador.

34. IA DOS PILOTOS

Pilotos IA devem adaptar comportamento conforme:

- posição no campeonato;
- ordem da equipe;
- rivalidade;
- confiança;
- condição de pista;
- risco necessário;
- contrato;
- pressão;
- estilo pessoal.

35. MUNDO PERSISTENTE

O jogo deve registrar acontecimentos históricos:

- campeões;
- vencedores;
- pódios;
- recordes;
- contratos;
- transferências;
- acidentes;
- estreias;
- aposentadorias;
- títulos de equipe;
- rivalidades.

Isso alimenta narrativa, notícias e legado.

36. NOTÍCIAS E NARRATIVA DINÂMICA

O jogo gera notícias com base em eventos reais do save. Não inventar manchetes desconectadas.

Exemplos:

- estreante surpreende;
- equipe rompe com fornecedor;
- piloto critica estratégia;
- patrocinador amplia contrato;
- rivalidade cresce;
- chefe técnico é contratado;
- atualização falha;
- chuva muda campeonato.

37. INTERFACE — PRINCÍPIO VISUAL

A interface deve parecer um software premium de gestão esportiva com linguagem de transmissão. Nada de excesso de brilho, telas vazias ou cartões sem hierarquia.

Princípios:

- informação prioritária sempre visível;
- no máximo 1 ação primária por bloco;
- cores com função semântica;
- gráficos legíveis;
- números alinhados;
- estados de hover/foco;
- animações curtas;
- transições suaves;
- modo responsivo;
- leitura adequada em desktop.

38. DIREÇÃO DE ARTE

As imagens criadas fora do Codex serão tratadas como assets oficiais do projeto. O Codex não deve inventar arte genérica no lugar delas quando existir asset definido.

Famílias de imagens futuras:

- capa;
- logo;
- menu principal;
- sede;
- garagem;
- pit wall;
- oficina;
- centro de engenharia;
- marketing;
- patrocinadores;
- pilotos;
- staff;
- circuitos;

- clima;
- pódio;
- celebração;
- telas de contratos;
- transmissão de corrida;
- loading;
- fundos;
- ícones;
- badges;
- molduras;
- cards.

39. ÁUDIO

Arquitetura deve prever:

- música de menu;
- trilha contextual;
- sons de UI;
- ambiente de paddock;
- motores;
- torcida;
- chuva;
- pit;
- alertas;
- vitória;
- derrota.

Mesmo quando os arquivos não existirem ainda, os gatilhos devem ficar preparados.

40. SAVE GAME

O save deve ser versionado e confiável.

Conteúdo:

- temporada;
- carreira;
- equipes;
- pilotos;
- staff;
- contratos;
- finanças;
- infraestrutura;
- desenvolvimento;
- campeonato;
- calendário;
- histórico;
- relações;
- reputação;
- patrocinadores;
- configurações.

Usar migração de schema para evitar quebrar saves com atualizações.

41. DIFICULDADE

A dificuldade deve alterar competência dos rivais, informação disponível, tolerância financeira e margem de erro, não simplesmente dar bônus oculto de velocidade.

Modos:

- acessível;
- normal;
- especialista;
- personalizado.

42. DELEGAÇÃO

Cada área pode ser delegada a um funcionário, com qualidade dependente da competência dele.

O jogador pode delegar:

- contratos;
- scouting;
- setup;
- P&D;
- estratégia;
- mídia;
- patrocínios;
- logística.

Delegar economiza atenção, mas não deve ser sempre a melhor opção.

43. ONBOARDING

Primeira carreira deve ter tutorial contextual e não um tutorial separado longo.

Sistema:

- checklist inicial;
- dicas acionáveis;
- explicações curtas;
- opção “saiba mais”;
- objetivos da primeira semana;
- primeira contratação;
- primeiro patrocinador;
- primeiro fim de semana de corrida.

44. ACESSIBILIDADE

- escalonamento de fonte;
- contraste;
- redução de movimento;
- não depender somente de cores;
- atalhos;

- tooltips;
- textos claros;
- confirmação para ações irreversíveis;
- opção de pausar animações.

45. PERFORMANCE

Metas:

- inicialização rápida;
- simulação desacoplada do DOM;
- renderização otimizada;
- listas virtualizadas quando necessário;
- dados carregados por módulos;
- imagens responsivas;
- cache;
- evitar loops pesados na UI;
- possibilidade futura de Web Worker para simulação.

46. TESTES

Testes mínimos:

- economia;
- contratos;
- progressão;
- pontuação;
- classificação;
- resultados;
- save/load;
- corrida;
- desgaste;
- incidentes;
- patrocinadores;
- desenvolvimento;
- IA de mercado.

Toda fórmula crítica deve ter testes determinísticos.

47. BALANCEAMENTO

Criar arquivo central de balanceamento. Nenhum valor importante deve ficar “mágico” em código.

Categorias:

- salários;
- custos;
- ganhos;
- evolução;
- desgaste;
- risco;
- performance;
- reputação;

- probabilidade;
- duração;
- moral.

48. REGRAS DE QUALIDADE PARA O CODEX

O Codex deve:

- ler esta Bíblia antes de qualquer alteração relevante;
- preservar sistemas já funcionais;
- evitar reescrever o projeto inteiro sem necessidade;
- trabalhar em módulos;
- testar cada sistema;
- não usar placeholders permanentes;
- não deixar botões falsos;
- não criar telas sem função;
- não duplicar lógica;
- documentar dados;
- tratar erros;
- manter responsividade;
- manter consistência visual;
- nunca sacrificar simulação para “parecer pronto”.

49. CONTRATO DE DESENVOLVIMENTO EM 3 PARTES

O jogo deve ser criado no máximo em três grandes entregas. A divisão não significa três protótipos. A Parte 1 já precisa ser um jogo funcional.

PARTE 1 — JOGO FUNCIONAL COMPLETO

Objetivo: entregar um loop de carreira jogável do início ao fim de uma temporada.

Obrigatório na Parte 1:

- criação de equipe;
- seleção de categoria;
- base de dados inicial;
- pilotos;
- staff;
- fabricantes/fornecedores;
- contratos;
- finanças;
- patrocinadores;
- calendário;
- circuitos;
- treinos;
- classificação;
- corrida 2D funcional;
- clima básico;
- pneus;
- setup simplificado;
- desgaste;
- incidentes;

- classificação do campeonato;
- IA rival;
- notícias básicas;
- reputação;
- save/load;
- temporada completa;
- tela de resultados;
- dashboard;
- interface responsiva;
- assets com fallback seguro;
- dados centralizados;
- testes essenciais.

Critério de aceite da Parte 1:

O usuário deve conseguir iniciar uma nova carreira, criar equipe, contratar pessoas, fechar parceiros, preparar a primeira etapa, disputar sessões, ver a corrida em 2D, receber resultado, avançar o calendário e completar a temporada sem depender de qualquer função falsa.

PARTE 2 — PROFUNDIDADE, IMERSÃO E REALISMO

Adicionar:

- P&D avançado;
- infraestrutura;
- scouting aprofundado;
- mercado dinâmico;
- relações;
- moral;
- ordens de equipe;
- rivalidades;
- imprensa;
- crises;
- marketing avançado;
- ativações;
- logística;
- lesões;
- contratos sofisticados;
- telemetria;
- setup avançado;
- clima dinâmico;
- estratégias avançadas;
- IA estratégica;
- histórico e recordes;
- premiações;
- eventos de diretoria;
- jovens;
- evolução de staff;
- melhorias visuais;
- áudio completo.

PARTE 3 — LAPIDAÇÃO AAA E LONGEVIDADE

Adicionar e finalizar:

- balanceamento fino;
- polimento de UX;
- animações;
- cinematografia;
- tela de transmissão premium;
- otimização;
- acessibilidade completa;
- tutorial;
- editor de dados quando viável;
- suporte a expansões;
- sistema robusto de temporadas futuras;
- aposentadorias;
- sucessão de talentos;
- recordes históricos;
- objetivos de legado;
- conquistas;
- analytics locais de balanceamento;
- QA;
- correção de bugs;
- preparação de release;
- documentação técnica final.

50. PROIBIÇÕES DA PARTE 1

Não aceitar:

- “demo” disfarçada;
- menus sem lógica;
- botões que dizem “em breve” para funções essenciais;
- resultados aleatórios sem simulação;
- temporada sem continuidade;
- corrida apenas decorativa;
- IA sem mercado;
- economia sem consequência;
- patrocinadores meramente cosméticos;
- save incompleto;
- dados críticos hardcoded na UI.

51. CRITÉRIO AAA

“AAA” aqui não significa orçamento de grande editora. Significa disciplina de produto:

- cada tela tem propósito;
- cada número tem origem;
- cada decisão tem consequência;
- cada ação possui feedback;
- cada sistema conversa com outros;
- visual tem hierarquia;
- carregamento é suave;

- erros são tratados;
- conteúdo parece parte do mesmo universo;
- o jogo respeita o tempo do jogador.

52. LOOP PRINCIPAL DE CARREIRA

Planejar → contratar → negociar → desenvolver → preparar etapa → correr → analisar → ajustar → crescer → construir legado.

Esse loop deve permanecer divertido em temporada 1 e temporada 10.

53. LOOP DE FIM DE SEMANA

Briefing → objetivos → clima → pneus → setup → treino → análise → ajustes → classificação → estratégia → corrida → entrevista → relatório técnico → finanças → recuperação.

54. LOOP DE CORRIDA

Observar gaps → analisar ritmo → comparar setores → ajustar instrução → responder a clima/incidente → gerenciar risco → receber feedback → tomar decisão → acompanhar consequência.

55. MODELO DE CONSEQUÊNCIA

Toda decisão relevante deve afetar ao menos uma variável persistente: dinheiro, moral, performance, reputação, confiança, relacionamento, risco, tempo, desenvolvimento ou mercado.

56. MATRIZ DE DEPENDÊNCIAS

Exemplos:

- salário maior → melhora chance de contratação → pressiona caixa;
- instalação melhor → acelera desenvolvimento → aumenta custo fixo;
- piloto popular → atrai patrocinador → aumenta pressão da mídia;
- estratégia agressiva → pode gerar posição → aumenta desgaste/risco;
- troca de fornecedor → potencial técnico → custo de adaptação;
- staff melhor → decisões melhores → custo e disputa no mercado.

57. GERADOR DE HISTÓRIAS EMERGENTES

O jogo deve registrar combinações de eventos e transformá-las em narrativa.

Exemplo: piloto jovem + três bons resultados + interesse rival + contrato terminando = arco de negociação.

Exemplo: atualização falha + duas quebras + patrocinador insatisfeito = crise técnica.

Exemplo: vitória na chuva + equipe pequena + piloto local = explosão de popularidade.

58. PAINÉIS PRINCIPAIS

- Home da equipe;
- Calendário;
- Caixa de entrada;
- Pilotos;
- Staff;
- Moto;

- Desenvolvimento;
- Fornecedores;
- Patrocínios;
- Finanças;
- Marketing;
- Scouting;
- Infraestrutura;
- Campeonato;
- Circuitos;
- Histórico;
- Corrida.

59. HOME DA EQUIPE

Deve mostrar em uma tela:

- próxima etapa;
- posição no campeonato;
- caixa;
- orçamento comprometido;
- moral;
- principais alertas;
- projetos;
- contratos a vencer;
- objetivo;
- notícias;
- tarefas.

60. SISTEMA DE ALERTAS

Prioridade:

- Crítico;
- Ação necessária;
- Importante;
- Informação.

Nunca sobrecarregar o usuário com dezenas de notificações iguais.

61. ARQUITETURA DE ESTADO

O estado do jogo deve ter store central ou módulos bem definidos. A UI nunca deve ser a fonte da verdade.

Exemplo de domínios:

career
world
team
riders
staff
contracts
finance
sponsors

technical
championship
calendar
race
news
settings

62. SIMULAÇÃO DETERMINÍSTICA OPCIONAL

Sempre que possível, o motor deve aceitar seed. Isso ajuda testes, reprodução de bugs e balanceamento.

63. EVENT BUS

Usar eventos internos para desacoplar sistemas.

Exemplos:

RACE_FINISHED
CONTRACT_SIGNED
SPONSOR_LOST
RIDER_INJURED
UPGRADE_COMPLETED
CHAMPIONSHIP_POSITION_CHANGED

64. LOG DE AUDITORIA DO SAVE

Guardar histórico técnico mínimo de ações e eventos para diagnóstico e geração narrativa.

65. PIPELINE DE ASSETS

Assets produzidos neste chat devem ter:

- nome padronizado;
- categoria;
- resolução;
- versão;
- fallback;
- referência no manifest.

Exemplo:

assets/garage/garage-main-v01.webp

66. MANIFESTO DE ASSETS

Criar asset-manifest.json para mapear fundos, ícones, logos, circuitos e imagens. A UI busca por chave lógica e não por caminho hardcoded.

67. LOCALIZAÇÃO

Preparar textos para i18n desde o início. PT-BR como idioma principal inicial. Strings de interface não devem ficar espalhadas pelo JavaScript.

68. EXPANSÕES FUTURAS

Arquitetura deve suportar:

- novas categorias;
- novos circuitos;

- novos países;
- novas temporadas;
- novos fabricantes;
- novos patrocinadores;
- novos pacotes de staff;
- campeonatos históricos;
- desafios.

69. MODDING FUTURO

Mesmo sem editor no lançamento, dados devem ser suficientemente externos para permitir futuro suporte a mods.

70. MÉTRICAS INTERNAS DE QUALIDADE

Durante desenvolvimento, validar:

- taxa de conclusão de temporada;
- equilíbrio financeiro;
- variedade de vencedores;
- distribuição de incidentes;
- taxa de renovação;
- competitividade da IA;
- diferença entre níveis de equipe;
- impacto de upgrades.

71. DEFINIÇÃO DE “PRONTO”

Uma funcionalidade está pronta somente quando:

- funciona;
- persiste no save;
- aparece corretamente na UI;
- afeta sistemas relacionados;
- possui estados vazios;
- trata erro;
- tem feedback;
- foi testada;
- não depende de placeholder.

72. ORDEM DE IMPLEMENTAÇÃO DA PARTE 1

- 1) scaffold e dados;
- 2) store;
- 3) carreira;
- 4) equipe;
- 5) pilotos/staff;
- 6) contratos;
- 7) finanças;
- 8) patrocinadores;
- 9) campeonato/calendário;
- 10) circuito;
- 11) motor de sessão;
- 12) corrida 2D;

- 13) IA;
- 14) resultados/pontuação;
- 15) save/load;
- 16) dashboard;
- 17) notícias;
- 18) QA e balanceamento.

73. PRIMEIRO MARCO JOGÁVEL

Antes de polir, garantir:

nova carreira → criar equipe → contratar piloto → contratar staff → escolher fornecedor → assinar patrocinador → entrar na etapa → classificar → correr → pontuar → salvar → carregar.

74. REGRA DE NÃO REGRESSÃO

A cada parte, sistemas anteriores permanecem funcionais. A Parte 2 não pode quebrar o loop completo da Parte 1; a Parte 3 não pode transformar sistemas profundos em decoração.

75. REGRA DE ENTREGA PARA O CODEX

Cada entrega do Codex deve vir acompanhada de:

- resumo do que foi implementado;
- arquivos alterados;
- sistemas concluídos;
- testes executados;
- limitações reais;
- próximos itens;
- instruções de execução.

76. PRÓXIMOS VOLUMES DESTA BÍBLIA

Este documento deve ser ampliado com:

- especificação matemática da simulação;
- modelo de contratos;
- modelo financeiro;
- banco de dados de entidades;
- esquema JSON;
- wireframes por tela;
- design system;
- roteiro de assets;
- IA detalhada;
- balanceamento;
- tutorial;
- acessibilidade;
- áudio;
- eventos;
- notícias;
- QA;
- checklist de release.

FIM DA FUNDAÇÃO — VOLUME INICIAL

Esta seção é obrigatória e deve ser tratada como a Constituição do projeto.

77. REGRA-MÃE DE IMERSÃO OPERACIONAL

O jogador não deve sentir que está administrando uma planilha com tema de corrida. Ele precisa sentir que está dentro do paddock, da garagem e do pit wall. Sempre que uma decisão puder ser representada visualmente por ambiente, personagem, animação, telemetria, rádio, painel ou ação de equipe, a interface deve preferir essa representação a um simples formulário.

A experiência deve alternar naturalmente entre três camadas:

- 1) Gestão estratégica — contratos, finanças, marketing, staff, fornecedores, planejamento e desenvolvimento.
- 2) Operação de fim de semana — garagem, briefing, treino, setup, pneus, classificação, grid e corrida.
- 3) Narrativa e consequência — reuniões, propostas, notícias, reações, rivalidades, patrocinadores, desempenho e legado.

A regra é: o jogador sempre deve saber “onde está”, “o que está acontecendo”, “quem está trabalhando nisso” e “qual consequência sua decisão terá”.

78. TUTORIAL GUIADO POR AVATAR — PRIMEIRA EXPERIÊNCIA

A primeira carreira deve ser conduzida por um avatar-guia persistente, apresentado como assessor executivo da nova equipe. Este personagem não é um pop-up genérico: ele deve ter identidade visual, nome configurável ou definido pelo jogo, retrato/ilustração, expressões e contexto narrativo.

Função do avatar:

- receber o jogador na criação da equipe;
- explicar apenas o necessário para a próxima decisão;
- apontar visualmente o botão, painel ou campo correto;
- explicar por que a decisão importa;
- confirmar a consequência após a ação;
- nunca bloquear o usuário com textos longos;
- oferecer botão “Entendi”, “Quero mais detalhes” e “Pular orientação”;
- lembrar conceitos importantes quando aparecem pela primeira vez;
- desaparecer progressivamente conforme o jogador domina o jogo.

Fluxo obrigatório do tutorial inicial:

- 1) boas-vindas e explicação do papel de proprietário;
- 2) criação da identidade da equipe;
- 3) escolha de país, base e filosofia;
- 4) definição de orçamento inicial;
- 5) escolha de fabricante/fornecedor disponível;
- 6) contratação do primeiro piloto;
- 7) contratação do núcleo mínimo de engenharia e mecânicos;
- 8) primeira proposta comercial e assinatura de patrocinador;
- 9) apresentação da sede e da garagem;

- 10) explicação do calendário;
- 11) chegada ao primeiro circuito;
- 12) briefing da etapa;
- 13) primeiro treino livre guiado;
- 14) primeira leitura de feedback técnico;
- 15) primeira alteração de setup;
- 16) classificação guiada;
- 17) apresentação do grid de largada;
- 18) corrida com instruções contextuais mínimas;
- 19) pós-corrida, relatório e consequências financeiras;
- 20) liberação completa do modo carreira.

Ao concluir esse ciclo, o jogador recebe mensagem clara: “Sua equipe agora é sua. Todas as áreas estão liberadas.” A partir daí, o avatar vira assistente opcional e só surge para novidades, alertas graves ou sistemas ainda não utilizados.

79. PROPOSTAS, NEGOCIAÇÕES E REUNIÕES IMERSIVAS

Contratos e propostas não devem aparecer apenas como tabelas. O sistema deve simular uma caixa de entrada executiva e reuniões.

Tipos de proposta:

- piloto oferecendo serviços;
- agente pedindo reunião;
- fabricante oferecendo pacote técnico;
- patrocinador interessado;
- patrocinador ameaçando saída;
- funcionário pedindo aumento;
- engenheiro rival demonstrando interesse;
- fornecedor oferecendo upgrade;
- investidor propondo aporte;
- organizador convidando a equipe para categoria superior;
- mídia solicitando entrevista;
- piloto pedindo status de número 1;
- fabricante exigindo prioridade a determinado piloto;
- parceiro comercial pedindo ativação de marca.

Cada proposta deve mostrar contexto, prazo, interesse da outra parte, pontos negociáveis, impacto provável e risco. Negociações devem permitir contraoferta e registrar memória do relacionamento. Propostas expiradas, rejeitadas ou mal conduzidas devem ter consequência futura.

80. GARAGEM VIVA — MECÂNICOS E ENGENHEIROS

A garagem deve funcionar como um ambiente operacional. A moto entra, é inspecionada, desmontada parcialmente, ajustada, preparada e liberada.

Mesmo em representação 2D/UI, o jogador deve perceber atividade real por meio de:

- mecânicos animados ou elementos visuais de trabalho;
- status por estação;

- cronômetros de serviço;
- peças sendo verificadas;
- checklist técnico;
- rádio interno;
- alertas do engenheiro;
- temperatura de pneus;
- desgaste visual/indicativo;
- disponibilidade de peças;
- status da moto “em pista / voltando / em box / em ajuste / pronta”.

Cada mecânico deve possuir impacto mensurável. Atributos recomendados:

- velocidade de trabalho;
- precisão;
- diagnóstico;
- experiência com fabricante;
- trabalho sob pressão;
- fadiga;
- comunicação;
- especialidade;
- risco de erro.

Um mecânico melhor pode reduzir tempo de intervenção, detectar defeitos antes que se tornem falhas e executar setup com menor chance de inconsistência. Equipe cansada ou inexperiente pode cometer pequenos erros reais de preparação.

81. SISTEMA DE TAREFAS DE GARAGEM

Toda alteração de moto deve gerar tarefa, duração e responsável. Exemplos:

- trocar relação;
- mudar configuração de suspensão;
- alterar altura;
- alterar geometria;
- trocar componentes de freio;
- substituir carenagem;
- revisar eletrônica;
- trocar mapa de motor;
- trocar pneus;
- reparar dano;
- checar corrente/transmissão;
- checar refrigeração;
- substituir componente desgastado.

O jogador pode priorizar tarefas quando o tempo de sessão é limitado. Se faltam oito minutos para o fim do treino e dois ajustes demorariam dez, ele precisa escolher. Isso deve criar tensão operacional real.

82. CONFIGURAÇÃO DA MOTO — MODO BÁSICO E AVANÇADO

O setup deve ser uma das áreas mais imersivas do jogo.

Modo básico:

- Estabilidade ↔ Agilidade
- Frenagem ↔ Entrada de curva
- Tração ↔ Rotação
- Aceleração ↔ Velocidade final
- Conservação de pneu ↔ Performance imediata
- Entrega suave ↔ Entrega agressiva

O jogo traduz o efeito em linguagem clara: “mais estável nas curvas rápidas”, “melhor saída de curva, porém maior desgaste”, “melhor velocidade de reta, mas aceleração inferior”.

Modo avançado pode expor parâmetros como:

- pré-carga dianteira/traseira;
- compressão;
- retorno;
- altura;
- geometria de direção;
- distribuição de rigidez;
- relação final;
- mapas de potência;
- freio-motor;
- controle de tração;
- anti-wheelie quando aplicável;
- gestão eletrônica;
- parâmetros aerodinâmicos permitidos;
- pressão/temperatura de pneus conforme abstração adotada.

Nem todo parâmetro precisa usar unidade real se isso prejudicar usabilidade; porém a relação causa-efeito precisa ser tecnicamente coerente.

83. FEEDBACK DE PILOTO E ENGENHEIRO

Após cada saída de pista, o piloto fornece feedback. O nível de precisão depende de seu atributo de feedback técnico e da qualidade do engenheiro.

Exemplos de feedback:

- “A frente fecha na entrada das curvas lentas.”
- “A moto está instável na frenagem forte.”
- “Falta tração na saída.”
- “Estou perdendo muito na reta.”
- “O pneu traseiro cai cedo demais.”
- “A moto demora para mudar de direção.”

O engenheiro deve transformar o relato em hipóteses e sugestões. Exemplo: “Podemos aliviar o freio-motor e revisar a distribuição da suspensão. Isso pode melhorar a entrada, mas aumentar a movimentação traseira.”

O jogador decide aceitar, ajustar manualmente ou ignorar. Nenhuma sugestão deve ser sempre correta: a qualidade depende do staff e dos dados disponíveis.

84. TREINOS LIVRES — EXPERIÊNCIA COMPLETA

Treino livre não é botão “simular”. Deve existir como sessão viva e útil.

Objetivos possíveis:

- aprender pista;
- encontrar setup;
- avaliar pneus;
- testar nova peça;
- simular classificação;
- simular stint de corrida;
- trabalhar ritmo constante;
- coletar dados;
- validar confiabilidade.

Antes da saída:

- selecionar pneu;
- combustível quando relevante;
- mapa;
- objetivo da saída;
- número planejado de voltas;
- setup;
- nível de risco.

Durante o treino, mostrar:

- mapa 2D com todos os competidores;
- posição relativa;
- tempos pessoais e gerais;
- setores;
- melhor volta;
- evolução da pista;
- tráfego;
- clima;
- desgaste;
- temperatura;
- alertas;
- comparação com companheiro;
- delta para referência.

Ao voltar aos boxes, o jogador recebe dados e feedback. Ajustes consomem tempo real da sessão. O relógio continua correndo enquanto mecânicos trabalham, salvo quando regras do jogo permitirem pausa por acessibilidade.

85. PLANO DE CORRIDAS DE TREINO

Cada saída da garagem deve ser um “run”. Registrar:

- run 1, run 2, run 3 etc.;
- setup utilizado;
- pneu;

- número de voltas;
- melhores setores;
- média de ritmo;
- queda de performance;
- comentários do piloto;
- alterações feitas depois.

A tela de análise deve permitir comparar runs lado a lado. Isso cria sensação real de engenharia e aprendizado.

86. CLASSIFICAÇÃO — SESSÃO AO VIVO

A classificação deve ser um dos momentos mais tensos do fim de semana.

Elementos obrigatórios:

- cronômetro regressivo central;
- mapa 2D ao vivo;
- tabela de tempos atualizando dinamicamente;
- posição provisória;
- setores com cores e deltas;
- indicação “volta lançada”, “volta de preparação”, “volta abortada”;
- trânsito na pista;
- risco de bandeira/incidente;
- evolução da pista;
- clima;
- corte para próxima fase quando houver múltiplas fases;
- destaque para piloto do usuário.

O jogador deve decidir quando sair dos boxes. Sair cedo oferece pista livre porém menos aderência; sair tarde pode oferecer pista melhor, porém aumenta risco de tráfego, bandeira amarela ou não conseguir abrir a volta a tempo.

A IA deve tomar decisões equivalentes. Não pode simplesmente gerar tempos aleatórios no fim.

87. VOLTA RÁPIDA E DELTAS

Durante classificação, cada piloto tem uma tentativa que evolui setor a setor. A UI deve mostrar:

- delta para pole provisória;
- delta para melhor volta pessoal;
- ganho/perda por setor;
- previsão de posição ao fim da volta;
- status de pneu;
- tráfego à frente.

Ao cruzar a linha, a posição deve atualizar imediatamente com animação de ranking. Se o piloto vai para P1, a UI precisa tornar isso um momento importante.

88. GRID DE LARGADA AO VIVO — REGRA INViolável

O grid precisa ser uma experiência própria, inspirada no nível de apresentação definido para o projeto F1 Vale Gold Edition, adaptada à motovelocidade.

Após a classificação, não mostrar apenas uma tabela. Criar uma apresentação de grid completa e dinâmica.

Elementos do grid:

- fundo cinematográfico do circuito/grid;
- pista ou reta principal reconhecível;
- posições desenhadas como slots reais de largada;
- ícone, miniatura ou avatar de cada piloto/moto;
- número do piloto;
- nome curto;
- equipe;
- fabricante;
- posição P1, P2, P3 etc.;
- tempo de classificação;
- diferença para pole;
- indicação de penalidade;
- pneu de largada quando aplicável;
- clima;
- temperatura da pista;
- contagem regressiva para largada.

O grid deve ser “vivo”: mecânicos presentes, motos sendo posicionadas, últimos ajustes, engenheiros conferindo dados, pneus sendo preparados, pilotos aguardando. Mesmo que parte seja representada por animação 2D, camadas ou sprites, a sensação deve ser de pré-largada real.

89. CERIMÔNIA DE GRID

Sequência recomendada:

- 1) tela “GRID DE LARGADA”;
- 2) apresentação da pole;
- 3) destaque da primeira fila;
- 4) rolagem ou montagem visual das demais posições;
- 5) foco nos pilotos da equipe do jogador;
- 6) resumo estratégico;
- 7) clima e pneus;
- 8) mensagem do engenheiro;
- 9) contagem regressiva;
- 10) transição cinematográfica para a largada.

O jogador pode pular a apresentação após já ter visto, mas a primeira vez deve ser exibida integralmente.

90. ÚLTIMAS DECISÕES NO GRID

Antes da largada, permitir decisões limitadas e realistas:

- estratégia de largada;

- agressividade inicial;
- mapa/entrega de potência;
- abordagem de pneus;
- instrução para cada piloto;
- reação a mudança súbita de clima;
- confirmação de setup permitido pelo regulamento.

Essas decisões precisam ter efeito real na simulação.

91. LARGADA — MOMENTO CINEMATOGRAFICO

A largada deve ter tratamento especial.

Sequência:

- motos alinhadas;
- câmera/mapa aproxima o grid;
- HUD reduzido;
- semáforo/contagem;
- reação dos pilotos calculada por atributo de largada, concentração, grip e risco;
- aceleração inicial;
- disputa até a primeira curva;
- reposicionamento dinâmico do ranking.

Uma boa ou má largada deve ser percebida imediatamente.

92. EFEITOS REAIS DO TRABALHO DOS MECÂNICOS

O trabalho da equipe técnica deve influenciar:

- confiabilidade;
- precisão de setup;
- velocidade de troca/reparo;
- chance de erro humano;
- qualidade de inspeção;
- detecção preventiva;
- tempo necessário entre runs;
- risco de soltar moto com problema não detectado.

Exemplo: um mecânico de suspensão excelente pode reduzir incerteza na alteração de setup. Um mecânico fatigado pode levar mais tempo ou realizar ajuste com pequena margem de erro. Um crew chief competente reduz retrabalho e melhora coordenação.

93. ERROS OPERACIONAIS PLAUSÍVEIS

Eventos possíveis, sempre raros e contextualizados:

- ajuste incorreto;
- sensor mal conectado;
- atraso em troca de componente;
- peça incompatível instalada e detectada antes da pista;
- falha de comunicação;
- aquecimento insuficiente de pneu;
- moto liberada alguns minutos depois do planejado.

Nunca usar esses eventos como punição arbitrária. Probabilidade deve depender de fadiga, competência, pressão e complexidade da tarefa.

94. PAINEL DE ENGENHARIA DURANTE SESSÕES

Painel lateral ou inferior deve concentrar:

- piloto 1 e piloto 2;
- estado da moto;
- pneus;
- desgaste;
- temperatura;
- último tempo;
- melhor tempo;
- posição;
- gaps;
- feedback atual;
- tarefa em garagem;
- tempo para conclusão;
- botão de chamar aos boxes;
- instruções de ritmo/risco;
- alertas críticos.

A interface não deve abrir dez janelas. O essencial fica acessível no mesmo ambiente de sessão.

95. RÁDIO E COMUNICAÇÃO DE EQUIPE

Criar feed de rádio textual e, futuramente, com áudio opcional.

Mensagens possíveis:

- “Box nesta volta.”
- “Temos tráfego a 3 segundos.”
- “Boa volta, P4 provisório.”
- “Temperatura traseira subindo.”
- “Precisamos de mais uma volta rápida.”
- “Chuva prevista em 6 minutos.”
- “Ajuste concluído, moto pronta.”

Mensagens devem ser geradas por estado real da simulação e não por frases aleatórias.

96. CHECKLIST PRÉ-SESSÃO

Antes de liberar “Ir para pista”, o jogo verifica:

- piloto disponível;
- moto operacional;
- pneus selecionados;
- setup válido;
- peças instaladas;
- combustível quando aplicável;
- estratégia mínima definida;

- nenhuma tarefa crítica pendente.

Se algo estiver faltando, o avatar ou engenheiro informa exatamente o que precisa ser resolvido.

97. EXPERIÊNCIA PÓS-CLASSIFICAÇÃO

Após a sessão:

- animação de resultado;
- posição final;
- comparação com expectativa;
- desempenho por setor;
- diferença para companheiro;
- reação do piloto;
- reação da equipe;
- impacto em moral/reputação;
- montagem do grid;
- penalidades aplicadas;
- recomendação estratégica para corrida.

98. EXPERIÊNCIA PÓS-CORRIDA

Depois da bandeirada, mostrar:

- classificação final;
- movimentos de posição;
- melhor volta;
- principais incidentes;
- desempenho do piloto;
- desempenho da moto;
- acerto de estratégia;
- desgaste;
- falhas/reparos necessários;
- pontos obtidos;
- campeonato atualizado;
- bônus de patrocinador;
- prêmio financeiro;
- moral;
- reputação;
- entrevistas/notícias;
- próximos compromissos.

99. REGRA DE LIBERDADE APÓS O TUTORIAL

O tutorial não deve transformar a carreira em trilho. Depois do primeiro ciclo completo, nenhuma tarefa deve ser obrigatoriamente guiada. O jogador pode navegar, delegar, simular sessões, assumir controle fino, ignorar recomendações ou gerir apenas áreas de interesse.

O sistema deve lembrar somente recursos nunca utilizados ou eventos críticos.

100. REQUISITO DA PARTE 1 — IMERSÃO NÃO É OPCIONAL

A Parte 1 já deve conter versão funcional de:

- avatar de tutorial;
- tutorial inicial completo;
- garagem operacional;
- tarefas de mecânicos;
- feedback de piloto;
- setup básico;
- treino livre com runs;
- classificação ao vivo;
- mapa 2D de sessão;
- deltas e ranking dinâmico;
- grid de largada visual e vivo;
- últimas decisões pré-largada;
- largada simulada;
- rádio/feed contextual;
- pós-sessão e pós-corrida.

A Parte 2 aprofunda esses sistemas. A Parte 3 lapida visual, animação, áudio, cinematografia e responsividade. Mas nenhum deles pode ficar ausente da primeira versão funcional.

101. REGRA DE REFERÊNCIA DE QUALIDADE — F1 VALE GOLD EDITION

A experiência de sessão deve seguir o mesmo princípio de qualidade adotado no projeto F1 Vale Gold Edition: transmissão esportiva, grid vivo, sensação de pit wall, informação em tempo real, hierarquia visual premium e decisões acontecendo dentro do contexto da prova.

A adaptação para motos deve respeitar particularidades da motovelocidade: dinâmica de piloto, aquecimento e degradação de pneus, sensibilidade de setup, risco de queda, comportamento em frenagem, tração, agilidade e influência direta do estilo do piloto.

102. REGRA DE AUDITORIA DE IMERSÃO

Antes de considerar treino, classificação, grid ou corrida prontos, responder:

- O jogador sente que está no paddock?
- Ele vê a equipe trabalhando?
- Ele entende por que a moto está melhor ou pior?
- O treino serve para aprender e decidir?
- A classificação produz tensão real?
- O grid parece uma apresentação de competição profissional?
- As decisões de setup têm consequência?
- Staff competente é perceptivelmente melhor?
- O avatar ensina sem incomodar?
- Após aprender, o jogador fica livre?

Se qualquer resposta for “não”, o sistema ainda não está pronto.

103. BLOCO TÉCNICO — MODELO DE COMPETIÇÕES E CATEGORIAS

A engine deve tratar cada categoria como um pacote de regras e parâmetros, nunca como lógica hardcoded. Uma categoria precisa declarar: id, nome exibido, nível de prestígio, tipo

de motocicleta, potência relativa, massa mínima quando aplicável, liberdade técnica, fornecedor único ou múltiplos fornecedores, número de motos por equipe, número de pilotos, formato de fim de semana, treinos, classificação, corrida principal, corrida sprint quando aplicável, pneus permitidos, quantidade de motores/componentes por temporada, regras de substituição, penalidades, pontuação, critérios de inscrição, teto ou referência de custos, premiação, reputação mínima e regras de promoção/entrada.

O motor deve suportar pelo menos quatro arquétipos: protótipo internacional de elite, protótipo intermediário, categoria formadora leve e superbike derivada de produção. Esses arquétipos servem como presets técnicos; os dados reais podem posteriormente mapear competições licenciadas sobre eles.

Cada campeonato deve ser definido por `season.json` e `championship.json`. Nenhuma regra esportiva importante deve depender de `if(nomeDoCampeonato === ...)`.

104. FORMATO DE FIM DE SEMANA CONFIGURÁVEL

Cada categoria declara uma sequência de `SessionDefinition`. Tipos mínimos: `PRACTICE`, `FREE_PRACTICE`, `PRE_QUALIFYING`, `QUALIFYING`, `QUALIFYING_GROUP`, `WARMUP`, `SPRINT`, `RACE`.

Cada sessão define: duração por tempo ou voltas, relevância para acesso à classificação, validade de melhores tempos, regra de pneus, regra de combustível, parque fechado, risco climático, valor de dados coletados, desgaste acumulado, limite de saídas de box, bandeiras e condição de encerramento.

A UI deve montar automaticamente o fim de semana lendo essa definição. Assim, uma competição com Q1/Q2, outra com classificação única e outra com duas corridas podem usar a mesma engine.

105. REGULAMENTO ESPORTIVO COMO DADOS

Criar `rulesEngine` com regras desacopladas. Tipos mínimos: `gridFormationRules`, `qualifyingRules`, `parcFerreRules`, `tyreRules`, `engineAllocationRules`, `componentAllocationRules`, `penaltyRules`, `pointsRules`, `replacementRules`, `redFlagRules`, `restartRules` e `classificationRules`.

Toda infração gera `SportsDecision` com: tipo, piloto/equipe afetado, sessão, motivo, severidade, penalidade, momento da aplicação, possibilidade de recurso e efeito no resultado/grid.

Penalidades suportadas: perda de posições no grid, largada do pit lane, adição de tempo, ride-through abstrato quando aplicável, long-lap abstrata quando aplicável, perda de pontos, desclassificação e advertência. O conjunto habilitado depende do regulamento da categoria.

106. RELAÇÃO FÁBRICA, SATÉLITE E CLIENTE

O sistema de fabricante precisa diferenciar níveis de suporte.

FACTORY: prioridade máxima, peças mais novas, engenheiros de fábrica, atualizações antecipadas, maior influência técnica, metas esportivas agressivas e exigências comerciais maiores.

SUPPORTED_SATELLITE: pacote competitivo, parte das atualizações, suporte técnico relevante, custo elevado e possível obrigação de desenvolver piloto ligado ao fabricante.

CUSTOMER: custo menor, especificação mais antiga ou limitada conforme regras, menor prioridade de peças e menor influência do fabricante.

INDEPENDENT_Tech: equipe compra ou desenvolve módulos permitidos e assume mais risco técnico.

O relacionamento fabricante-equipe possui rating de 0–100, confiança, importância estratégica, resultados, cumprimento de metas e valor comercial. Esse relacionamento influencia disponibilidade de contrato e qualidade do pacote oferecido.

107. CONTRATO DE FORNECIMENTO DE MOTO

SupplierContract deve suportar: manufacturerId, teamId, seasonStart, seasonEnd, supportTier, annualFee, perBikeFee, performanceBonus, minimumChampionshipTarget, exclusivity, riderClause, technicalStaffIncluded, upgradePriority, sparePartsAllowance, testingSupport, dataSharingLevel, marketingObligations, terminationClause e renewalOption.

A negociação deve calcular attractivenessScore com reputação da equipe, resultados recentes, orçamento, piloto principal, mercado do país, patrocinadores, histórico com fabricante e vagas disponíveis.

Fabricantes não podem aceitar número ilimitado de clientes. Cada um possui capacityFactoryTeams, capacitySupportedTeams e capacityCustomers.

108. PNEUS — MODELO TÉCNICO

TyreCompound deve possuir: id, tipo seco/molhado, gripBase, warmupRate, idealTempMin, idealTempMax, degradationRate, cliffThreshold, wetGrip, aquaplaningResistance, stability, feedbackSensitivity e durability.

Estado do pneu por roda: wear 0–100, temperature, pressureIndex abstrato, gripMultiplier, heatCycles, damageRisk.

Grip efetivo deve considerar composto × temperatura × desgaste × pista × chuva × estilo do piloto × setup × agressividade.

O pneu nunca deve perder desempenho de forma linear perfeita. Usar curva com fase inicial, janela ideal, degradação progressiva e possível cliff. O perfil da curva varia por composto.

109. MODELO MATEMÁTICO DE PERFORMANCE — PRINCÍPIO

Nunca calcular volta com uma única nota geral. O tempo é soma de microsegmentos.

Para cada microsegmento s:

$segmentTime = baseSegmentTime \times performanceFactor$.

performanceFactor nasce de bikeFit, riderFit, tyreFactor, setupFactor, trackEvolution, weatherFactor, trafficFactor, fatigueFactor, damageFactor e controlledVariance.

Todos os fatores devem ser normalizados próximos de 1.0. Valores menores melhoram o tempo; valores maiores pioram. Limitar cada termo por clamp para impedir resultados absurdos.

Exemplo conceitual:

performanceFactor = clamp(1 + bikeDelta + riderDelta + tyreDelta + setupDelta + weatherDelta + trafficDelta + fatigueDelta + damageDelta + variance, minFactor, maxFactor).

O balanceamento real fica em balance/race-model.json.

110. BIKE FIT POR TIPO DE TRECHO

Cada moto possui vetor técnico normalizado: acceleration, topSpeed, braking, cornerEntry, midCorner, cornerExit, agility, stability, traction, electronics, aeroEfficiency, tyreManagement e reliability.

Cada microsegmento possui pesos equivalentes. bikeFit = produto ponderado entre os atributos da moto e os pesos daquele trecho.

Exemplo: reta longa valoriza topSpeed, acceleration e aeroEfficiency; hairpin valoriza braking, cornerEntry e traction; sequência rápida valoriza agility, stability e aeroEfficiency.

Isso deve permitir uma moto dominar certas pistas e sofrer em outras sem bônus artificiais por circuito.

111. RIDER FIT E ESTILO DO PILOTO

Cada piloto possui atributos técnicos e estilo. O desempenho em microsegmento deve usar speed, braking, cornerEntry, midCorner, cornerExit, overtaking, defence, wetSkill, tyreManagement, consistency, adaptability e riskControl.

Estilos mínimos: lateBraker, smooth, aggressive, cornerSpeed, tractionFocused, balanced.

O estilo interage com setup e moto. Um piloto agressivo pode extrair mais de uma moto estável, porém desgastar pneu; um piloto suave pode preservar composto, mas perder temperatura em condições frias.

Não usar estilo como bônus fixo. Ele modifica pesos e riscos.

112. SETUP SCORE SEM RESPOSTA ÓBVIA

Cada circuito gera uma janela ideal multidimensional, nunca um único setup perfeito universal. O piloto também desloca essa janela.

setupScore é calculado pela distância entre setup atual e preferredWindow para: frontStability, rearTraction, turning, brakingSupport, powerDelivery, gearing, electronics e aeroBalance quando aplicável.

O jogador não vê os números ideais. Ele descobre a região através de feedback, telemetria e runs.

A qualidade do engenheiro reduz a incerteza. A qualidade do piloto melhora a precisão do feedback. Isso transforma treino em processo de descoberta.

113. MODELO DE CONFIABILIDADE

Cada componente possui health, wearRate, baseFailureRate, temperatureSensitivity, crashSensitivity e inspectionConfidence.

FailureRisk por tick deve considerar: saúde do componente, quilometragem, temperatura, qualidade da manutenção, erro de montagem, agressividade de uso e confiabilidade de projeto.

Falhas possíveis em níveis: warning, performanceLoss, critical, terminal.

A UI deve permitir detectar sintomas antes de falha total quando sensores, engenheiros e mecânicos forem competentes. O jogador pode continuar na pista assumindo risco.

114. MODELO DE QUEDA E ERRO

Queda não pode ser RNG puro. Criar incidentPressure baseado em riderRisk, currentPaceDemand, gripAvailable, tyreState, weather, trafficDensity, fatigue, damage, trackRisk e mentalPressure.

Quando incidentPressure passa faixas de risco, a chance de erro aumenta. Resultados progressivos: pequena correção, perda de trajetória, perda de tempo, saída de pista, contato e queda.

Pilotos com riskControl alto reduzem escalada do incidente. Agressividade alta aumenta ganho potencial e incidentPressure.

115. ULTRAPASSAGEM POR OPORTUNIDADE

Cada microsegmento declara overtakingOpportunity 0–1. Uma tentativa é criada somente se gap, paceDelta e trecho permitirem.

AttackScore = atacante: overtaking + braking/traction relevante + confiança + pneu + ritmo + agressividade.

DefenceScore = defensor: defence + trecho + pneu + estabilidade + confiança.

O resultado não deve ser binário simples. Possíveis estados: closeIn, showMove, attempt, completed, defended, counterAttack, contact, mistake.

A UI 2D deve refletir isso aproximando ícones e gerando feed contextual.

116. IA ESTRATÉGICA EM CAMADAS

A IA deve usar três horizontes.

Season AI: orçamento, contratos, pilotos, staff, fabricante, P&D, infraestrutura e metas.

Weekend AI: preparação, objetivos, alocação de pneus, plano de treino, setup e estratégia.

Live Session AI: timing de saída, ritmo, risco, reação a clima, chamada ao box e defesa/ataque.

Cada equipe possui competence e personality. A IA mais fraca comete erros plausíveis de análise e timing; não recebe desvantagem artificial de velocidade.

117. ARQUITETURA DE DADOS — ENTIDADES OBRIGATÓRIAS

Criar IDs estáveis e relações por ID. Entidades mínimas: GameConfig, Career, Season, Championship, Event, Circuit, TrackLayout, SessionDefinition, Team, Manufacturer, Supplier, BikeSpec, BikeInstance, Component, Rider, StaffMember, StaffRole, Contract, Sponsor, SponsorContract, TyreCompound, TyreSet, Setup, Run, Session, SessionEntry, Lap, SectorTime, Incident, Penalty, WeatherState, DevelopmentProject, Facility, FinanceTransaction, NewsItem, Relationship, Objective e Achievement.

Nunca duplicar objetos completos dentro de outros objetos se uma referência por ID resolver. Save deve ser serializável e migrável.

118. SCHEMA JSON — EXEMPLO DE RIDER

Estrutura conceitual:

```
{
  "id": "rider_x",
  "name": "...",
  "nationality": "...",
  "age": 22,
  "attributes": {
    "speed": 78,
    "qualifying": 74,
    "racePace": 80,
    "braking": 82,
    "cornerEntry": 76,
    "midCorner": 79,
    "cornerExit": 81,
    "wetSkill": 68,
    "tyreManagement": 77,
    "overtaking": 75,
    "defence": 72,
    "consistency": 73,
    "feedback": 70,
    "riskControl": 69
  },
  "personality": [],
  "style": "balanced",
  "reputation": 0,
```

```
"potential": 0,  
"marketability": 0  
}
```

Atributos exibidos ao usuário podem sofrer scoutingFog. O valor real fica interno.

119. SCHEMA JSON — CIRCUITO E MICROSEGMENTOS

Circuit deve ter metadata e layouts. TrackLayout deve conter um path visual normalizado e segments.

Cada segment possui: id, order, type, lengthWeight, baseTimeWeight, curvature, speedClass, brakingDemand, tractionDemand, aeroDemand, overtakingOpportunity, crashRisk, tyreStressFront, tyreStressRear, wetSensitivity e coordinates/normalizedProgress para renderização 2D.

O mesmo modelo alimenta simulação e movimento visual dos ícones. Nunca manter um traçado visual sem ligação com os segmentos usados pela engine.

120. GRID, CLASSIFICAÇÃO E RESULTADO — FONTE ÚNICA DE VERDADE

GridEntry deve ser gerado pelo rulesEngine a partir de resultados da classificação + penalidades + regras do campeonato. A UI nunca calcula grid por conta própria.

Campos: gridPosition, riderId, teamId, qualifyingPosition, qualifyingTime, penaltyAdjustment, pitLaneStart, tyreStart, status e notes.

QualifyingResult, GridEntry e RaceResult devem ser objetos separados. Isso evita erros quando um piloto faz P3 na classificação, recebe penalidade e larga P8.

121. MOTOR DE SESSÃO — STATE MACHINE

Estados mínimos: PRE_SESSION, GARAGE_READY, GREEN, YELLOW_LOCAL, RED, CHECKERED, POST_SESSION, CLOSED.

Cada piloto possui subestado: IN_GARAGE, PIT_OUT, OUT_LAP, TIMED_LAP, IN_LAP, PIT_IN, STOPPED, RETIRED, FINISHED.

A transição deve ser explícita. Isso simplifica UI, rádio, regras e testes.

122. TELEMETRIA COMO DADOS DERIVADOS

Não armazenar tudo como valores arbitrários. Telemetria visual deve ser derivada do estado da simulação: sectorDelta, paceTrend, tyreTrend, reliabilityTrend, trafficStatus, weatherRisk, setupConfidence e raceProjection.

A interface pode simplificar, mas não inventar. Se o painel diz “pneu traseiro degradando rápido”, isso precisa vir da curva real de desgaste.

123. FINANÇAS — LEDGER CONTÁBIL

Toda movimentação monetária gera FinanceTransaction com date/gameDay, category, sourceId, amount, currency, status, recurringReference e description.

Dashboard financeiro deriva do ledger. Não manter totais independentes sem reconciliação.

Categorias mínimas: sponsorIncome, prizeIncome, investment, merchandise, salaries, bonuses, supplierFees, parts, travel, logistics, facilities, development, insurance, penalties, termination e operations.

Fluxo de caixa, orçamento comprometido e projeção são cálculos distintos.

124. CONTRATOS — ENGINE ÚNICA

Rider, staff, sponsor e supplier usam ContractCore compartilhado: parties, start, end, baseValue, bonuses, clauses, status, negotiationState e termination.

Cada tipo adiciona módulos específicos. Isso reduz duplicação e permite IA negociar com a mesma base.

Estados: DRAFT, OFFERED, COUNTERED, ACCEPTED, SIGNED, ACTIVE, EXPIRING, EXPIRED, TERMINATED.

125. NEGOCIAÇÃO COM MEMÓRIA

Cada contraparte possui negotiationMemory: propostas recentes, rejeições, concessões, promessas, confiança e paciência.

Ofertas repetidamente abusivas reduzem willingnessToNegotiate. Boas relações podem flexibilizar exigências. O jogador deve perceber isso por texto e indicadores, não por fórmula exposta.

126. GERAÇÃO DE PILOTOS E STAFF FUTUROS

Para carreiras longas, criar youthGenerator. Entradas: país, tradição motociclística, investimento de base, categoria, ano e seed.

Gerar atributos coerentes, potencial, personalidade, estilo e reputação inicial. Evitar clones estatísticos. Distribuições devem criar especialistas, pilotos completos e talentos inconsistentes.

Staff também deve envelhecer, evoluir, mudar de função e se aposentar.

127. PROGRESSÃO E DECLÍNIO

Pilotos possuem developmentCurve dependente de idade, potencial, quilometragem, qualidade de equipe, moral, lesões, resultados e adaptação.

A evolução não pode ser +1 automático anual. Atributos podem subir, estabilizar ou cair de forma diferente.

Experiência melhora consistência, feedback e gestão; idade avançada pode reduzir velocidade pura ou recuperação física, mas aumentar leitura de corrida.

128. P&D — FILA DE PROJETOS

DevelopmentProject possui domain, targetAttribute, complexity, baseCost, duration, staffAllocation, facilityRequirement, risk, expectedGainRange, hiddenActualGain, sideEffectChance e status.

Estados: IDEA, RESEARCH, DESIGN, VALIDATION, MANUFACTURING, READY, FAILED, CANCELLED.

O jogador escolhe prioridades. Projetos simultâneos disputam staff e instalações.

129. PARTES, ESTOQUE E LOGÍSTICA

Criar Inventory. Componentes têm quantity, reserved, inTransit, damaged e installed.

Uma queda pode destruir componente e gerar custo real. Uma equipe sem peça sobressalente pode perder sessão. Logística ruim pode atrasar entrega de atualização.

O Codex deve evitar transformar isso em microgestão excessiva: interface oferece recomendações e níveis mínimos automáticos, mas o sistema existe de verdade.

130. UI — MAPA DE TELAS OBRIGATÓRIO

Fluxo principal: Splash → Menu → Nova Carreira/Carregar → Criação da Equipe → Dashboard → Inbox → Pilotos → Staff → Moto/Setup → Engenharia/P&D → Fornecedores → Patrocínios → Marketing → Finanças → Infraestrutura → Calendário → Evento → Garagem → Treino → Análise → Classificação → Resultado da Classificação → Grid Ao Vivo → Corrida → Resultado → Pós-Corrida → Campeonato/Histórico.

Cada tela deve declarar entrada, dados mostrados, ações, estados vazios, alertas, retorno e persistência.

131. COMPONENTES UI REUTILIZÁVEIS

Criar componentes lógicos reutilizáveis: TopBar, SideNav, ContextHeader, StatusBadge, StatCard, DataTable, DriverCard, StaffCard, ContractCard, SponsorCard, BikeStatusPanel, TyreWidget, SetupSlider, WeatherWidget, Timeline, LiveTimingTable, TrackMap, RadioFeed, GarageTaskCard, GridSlot, ResultRow, Modal, Tooltip, ConfirmDialog, Toast e AvatarGuide.

Evitar HTML duplicado entre telas.

132. LIVE TIMING — ESPECIFICAÇÃO

Tabela de timing precisa suportar posição, número, piloto, equipe, melhor tempo, gap, último tempo, setor 1/2/3, volta atual, estado de pista, pneu, voltas do pneu e status.

Atualizações devem ser incrementais. Durante volta rápida, setor muda assim que cruzado. Ranking atualiza quando tempo é completado.

Usuário pode filtrar “todos”, “minha equipe” ou “batalha próxima”.

133. TRACK MAP — ESPECIFICAÇÃO VISUAL E TÉCNICA

Renderizar SVG ou Canvas a partir do path normalizado do TrackLayout. Ícones percorrem progress 0–1 mapeado ao path.

A posição visual vem do estado da simulação, nunca de animação independente. Se a simulação pausa, os ícones pausam. Se velocidade de jogo muda, animação acompanha.

Permitir zoom/foco em piloto, destaque de batalha, bandeiras por setor e status de box.

134. VELOCIDADE DA SIMULAÇÃO

Controles mínimos: pausa, 1x, 2x, 4x, 8x. Em momentos críticos, opção de auto-pause configurável: acidente do piloto, mudança forte de chuva, entrada na janela crítica de classificação, alerta mecânico grave e última volta.

Aceleração não pode alterar resultados. Somente o tempo de execução visual.

135. SAVE — VERSIONAMENTO E MIGRAÇÃO

SaveGame contém schemaVersion. Cada release com mudança estrutural adiciona migrateSave(previousVersion).

Nunca descartar carreira por mudança de schema sem tentativa de migração. Criar backup local antes de migrar quando possível.

136. TESTES AUTOMATIZADOS DO MOTOR

Criar testes para: mesmo seed = mesmo resultado; mudança de setup altera distribuição esperada; pneu desgastado piora performance; chuva favorece wetSkill; moto de reta ganha nos segmentos de reta; penalidade altera grid corretamente; contrato expira; ledger fecha; save round-trip preserva estado; IA não gasta além das regras; classificação respeita cronômetro; bandeira vermelha interrompe sessão; grid não duplica posições.

137. SIMULAÇÃO EM LOTE PARA BALANCEAMENTO

Criar modo developer que roda centenas ou milhares de corridas sem UI e gera métricas: vitória por equipe, DNF rate, crash rate, gap médio, efeito de atributo, vantagem por fabricante, desgaste médio e distribuição de posições.

Esse modo é obrigatório para calibrar equilíbrio sem depender de teste manual.

138. REGRA ANTI-CHEAT DA IA

A IA só pode usar informação que estaria disponível para uma equipe equivalente. Forecast de clima da IA depende do staff; scouting depende da rede; condição da moto rival não pode ser conhecida exatamente; contratos de outros times só são conhecidos se públicos ou vazados por evento.

139. REGRA DE OBSERVABILIDADE

Toda fórmula crítica deve poder gerar debugExplain em modo desenvolvedor: quais fatores contribuíram para tempo de volta, falha, queda, decisão de IA, valor de contrato ou interesse de patrocinador.

O debug não aparece ao jogador normal. Serve para o Codex testar e corrigir sistemas sem adivinhar.

140. DEFINITION OF DONE — BLOCO TÉCNICO

Este bloco só estará implementado quando a Parte 1 possuir: engine de competição configurável por dados; sessões via state machine; grid derivado do rulesEngine; microsegmentos de pista; modelo de performance multifatorial; pneus com temperatura/desgaste; setup com efeito mensurável; confiabilidade; incidentes contextualizados; IA em fim de semana; contratos centralizados; ledger financeiro; saves versionados; TrackMap ligado à simulação; LiveTiming funcional; testes determinísticos; modo de simulação em lote.

Essas estruturas são fundação. Partes 2 e 3 podem aumentar profundidade e apresentação, mas não substituir a arquitetura por atalhos.

141. REGRA-MÃE — PILOTOS FICTÍCIOS E IDENTIDADE PRÓPRIA

O elenco principal do jogo deve ser formado prioritariamente por pilotos fictícios originais. Não usar pilotos oficiais como base obrigatória da carreira. Cada personagem deve possuir nome próprio, rosto original, número, nacionalidade, biografia, personalidade, estilo de pilotagem, reputação, histórico esportivo e identidade visual exclusivos.

A intenção é criar um universo crível de motovelocidade com liberdade criativa, longevidade, segurança jurídica e maior capacidade narrativa. Se no futuro houver conteúdo real licenciado, ele deve entrar como pacote de dados opcional e separado, sem quebrar a carreira fictícia.

142. REGRA ANTI-CÓPIA E ANTI-SEMELHANÇA INDEVIDA

Pilotos fictícios não podem ser cópias disfarçadas de pilotos reais. Evitar combinações que tornem a associação imediata: nome quase idêntico, mesma nacionalidade, mesmo número, aparência muito semelhante, trajetória equivalente, apelido parecido ou personalidade espelhada de atleta real.

Antes de aprovar um personagem, auditar: nome, rosto, número, nacionalidade, biografia, equipe, capacete e arquétipo. Se a combinação lembrar excessivamente uma pessoa real identificável, o personagem deve ser redesenhado.

143. PADRÃO DE NOMES FICTÍCIOS

O banco de dados deve usar nomes plausíveis e culturalmente coerentes, mas originais. Distribuir o elenco entre diferentes países e regiões, incluindo Brasil, Argentina, Portugal, Espanha, Itália, França, Reino Unido, Alemanha, Japão, Austrália, Estados Unidos, África do Sul, Tailândia, Turquia e mercados emergentes.

Regras: permitir nome completo e nome curto de transmissão; evitar nomes famosos ou quase idênticos; permitir apelido opcional; manter pronúncia plausível; evitar repetição excessiva de sobrenomes; usar variedade cultural realista.

144. DIRETRIZ VISUAL — RETRATOS 4K ULTRARREALISTAS

As fotos dos pilotos fictícios serão produzidas aqui no chat e tratadas como assets oficiais do projeto. O Codex não deve substituir um retrato oficial por imagem genérica quando o asset existir.

Padrão obrigatório: visual 4K sempre que viável, ultrarrealista, iluminação profissional, textura natural de pele/cabelo/tecido/couro, enquadramento esportivo premium, paddock ou box desfocado quando adequado, nitidez facial alta, sem aspecto de banco de imagens e sem copiar fotografias de pessoas reais.

145. PACOTE MÍNIMO DE ASSETS POR PILOTO

Cada piloto deve possuir:

- 1) retrato principal oficial;
- 2) retrato meio-corpo/paddock;
- 3) avatar pequeno para UI;
- 4) recorte transparente ou limpo para cards;
- 5) imagem opcional com capacete;
- 6) imagem opcional de comemoração/pódio;
- 7) imagem opcional promocional de temporada.

A identidade facial precisa permanecer consistente entre todas as imagens do mesmo personagem.

146. FICHA DE IDENTIDADE COMPLETA DO PILOTO

Campos mínimos:

- id;
- nome completo;
- nome curto;
- apelido;
- nacionalidade;
- cidade/região de origem;
- idade;
- data de nascimento;
- número principal;
- número alternativo;
- idioma principal;
- categoria atual;
- equipe atual;
- agente/empresário;
- reputação;
- popularidade;
- moral;
- forma atual;
- valor de mercado;
- salário esperado;
- portraitMain;
- portraitPaddock;
- avatar;
- helmetStyleId;

- signatureColor;
- biographyShort;
- biographyMedium;
- biographyLong.

147. ATRIBUTOS DE PILOTAGEM — MODELO AVANÇADO

Cada piloto deve possuir atributos separados, não apenas nota geral:

rawSpeed, qualifying, racePace, start, braking, cornerEntry, midCorner, cornerExit, tractionFeel, wetSkill, tyreManagement, consistency, overtaking, defence, aggression, composure, riskControl, adaptability, technicalFeedback, stamina, recovery e crashTendency.

A nota geral, se exibida, deve ser derivada desses atributos e variar conforme contexto/categoria. Nunca usar overall como fonte principal da simulação.

148. ATRIBUTOS HUMANOS, MENTAIS E COMERCIAIS

Adicionar ambition, loyalty, patience, ego, professionalism, teamwork, leadership, mediaSkill, sponsorAppeal, fanAppeal, pressureResistance, contractFlexibility e transferInterest.

Esses fatores impactam negociações, moral, entrevistas, conflitos, patrocinadores, comportamento em equipe e decisões de carreira.

149. ARQUÉTIPOS DE PILOTO

Criar arquétipos combináveis: prodígio explosivo, veterano técnico, classificador puro, piloto de corrida, especialista em chuva, conservador estratégico, agressivo de ultrapassagem, estrela midiática, desenvolvedor de moto, talento inconsistente, líder de equipe, segundo piloto ideal, piloto pagante/comercial, promessa de base e veterano em busca do último contrato.

Arquétipo não concede bônus mágico. Ele modifica pesos de decisão, comportamento, personalidade e desenvolvimento.

150. BIOGRAFIAS E HISTÓRIA DE VIDA

Cada piloto deve ter biografia curta, média e longa. A história pode envolver origem humilde, tradição familiar, destaque em categorias de base, lesão, queda de rendimento, retorno, ascensão tardia, carreira comercial, promessa nacional ou reconstrução em equipe pequena.

A biografia deve conversar com resultados e carreira. O jogo não pode continuar descrevendo um piloto como “promessa desconhecida” depois que ele já virou campeão mundial.

151. VOZ NARRATIVA INDIVIDUAL

Cada personagem precisa responder de maneira coerente com sua personalidade. Piloto técnico fala de setup e dados; piloto agressivo cobra ataque; piloto reservado é objetivo; astro midiático gera declarações fortes; veterano usa experiência.

Aplicar isso em feedback técnico, entrevistas, rádio textual, reuniões, reclamações contratuais, reações a vitórias/derrotas e mensagens internas.

152. CAPACETE, NÚMERO E IDENTIDADE PESSOAL

Cada piloto deve ter número preferencial, número alternativo, estilo de capacete, cor pessoal secundária, monograma opcional e assinatura visual. Mesmo dentro da mesma equipe, deve ser imediatamente reconhecível no grid, timing, cards e notícias.

153. STAFF FICTÍCIO COM A MESMA FILOSOFIA

Team principals, engenheiros, mecânicos-chefes, diretores comerciais, scouts, fisioterapeutas e demais personagens importantes também devem ser fictícios e originais, com nome, rosto, atributos, personalidade e retratos próprios.

Staff principal deve possuir retrato 4K oficial, retrato de ambiente de trabalho e avatar de UI quando viável.

154. MERCADO DE PILOTOS E AGENTES

Pilotos devem possuir representante/agente, objetivos de carreira e preferências. A negociação considera salário, bônus, papel na equipe, duração, competitividade esperada, relação com fabricante, prioridade técnica, obrigações de mídia, cláusulas de saída e reputação da estrutura.

Jovens podem aceitar projetos menores com perspectiva de crescimento. Estrelas exigem desempenho, salário, protagonismo e estrutura.

155. ESTRUTURA DE CONTRATO DE PILOTO

RiderContract deve adicionar ao ContractCore: riderId, teamId, role, baseSalary, pointsBonus, podiumBonus, winBonus, titleBonus, mediaObligations, performanceClause, releaseClause, buyout, manufacturerClause, numberOneStatus, technicalPriority, renewalOption e agentCommission.

Promises devem ser persistidas. Se a equipe promete prioridade e não cumpre, confiança e disposição para renovar diminuem.

156. GERAÇÃO DO ELENCO INICIAL

O banco inicial deve conter titulares, reservas, pilotos de teste, jovens promessas, veteranos livres, talentos regionais, pilotos em recuperação e pilotos comerciais. O objetivo é existir mercado suficiente para contratação, substituição e scouting sem sensação de lista vazia.

O mundo deve manter talentos fora da categoria principal para permitir ascensão natural ao longo das temporadas.

157. PROGRESSÃO, PICO E DECLÍNIO

O desenvolvimento depende de idade, potencial, categoria, quilometragem, resultados, qualidade do staff, moral, lesões, confiança, adaptação e ambiente da equipe.

Atributos evoluem separadamente. Um veterano pode perder velocidade pura e ganhar consistência/feedback; um jovem pode ganhar ritmo rapidamente e continuar inconsistente; uma lesão pode afetar recuperação, confiança e agressividade.

158. FAMA E PRESENÇA MIDIÁTICA

Níveis sugeridos: desconhecido, promessa local, talento observado, respeitado no paddock, estrela nacional, celebridade internacional e ícone veterano.

Fama interfere em salário, patrocínio, venda de produtos, pressão, seguidores, solicitações de mídia, interesse de equipes rivais e valor comercial.

159. INTEGRAÇÃO DOS RETRATOS COM A UI

O retrato não é decoração. Usar imagem do piloto no perfil, scouting, mercado, contrato, notícias, lineup, classificação, grid ao vivo, resultado, pódio, entrevista e eventos narrativos.

O LiveTiming pode usar avatar pequeno; o grid pode usar retrato recortado; notícias usam portraitMain; negociação usa portraitPaddock; pódio usa celebration quando disponível.

160. PIPELINE E NOMENCLATURA DOS ASSETS

Estrutura recomendada:

assets/riders/<rider-id>/portrait-main-v01.webp

assets/riders/<rider-id>/portrait-paddock-v01.webp

assets/riders/<rider-id>/avatar-v01.webp

assets/riders/<rider-id>/helmet-v01.webp

assets/riders/<rider-id>/celebration-v01.webp

asset-manifest.json deve mapear cada chave lógica. Registrar versão, resolução, uso e fallback.

161. REGRA DE CONSISTÊNCIA VISUAL ENTRE TEMPORADAS

O mesmo piloto não pode mudar aleatoriamente de rosto, idade aparente, cabelo, barba, pele ou biotipo. Mudanças precisam ser narrativas e deliberadas: envelhecimento, novo visual, lesão, mudança de equipe ou atualização oficial do asset.

Sempre que criar nova imagem do mesmo personagem aqui no chat, usar como referência visual as imagens previamente aprovadas daquele piloto.

162. DEFINITION OF DONE — BLOCO DE PILOTOS FICTÍCIOS

Este bloco só estará concluído quando a Parte 1 possuir: banco inicial de pilotos fictícios; nomes originais; IDs estáveis; fichas completas; atributos técnicos/humanos; contratos; agentes; mercado funcional; retratos integrados à UI; asset-manifest preparado; regra de consistência visual; e nenhuma dependência de piloto oficial real.

A direção visual de produção deve considerar os retratos 4K ultrarrealistas como parte essencial da identidade AAA do jogo, e não como polimento opcional de fase futura.

163. REGRA-MÃE DE PLATAFORMA — MOBILE-FIRST HORIZONTAL

O foco principal do jogo é mobile, mas a mesma base deve ser plenamente jogável em PC. Toda decisão de UX, densidade de informação, tamanho de controles, hierarquia visual e desempenho deve nascer primeiro pensando em smartphones e depois escalar para telas maiores.

No celular, a orientação oficial é horizontal/landscape. A experiência principal não deve ser desenhada para modo retrato. Se o usuário abrir o jogo em orientação vertical, exibir uma tela premium de orientação com mensagem clara solicitando que gire o aparelho. Não renderizar a interface estratégica complexa comprimida em retrato.

O jogo deve usar CSS responsivo, unidades fluidas, clamp(), safe-area-inset, touch targets confortáveis e breakpoints orientados por espaço real disponível, não apenas por modelo de aparelho.

164. FULLSCREEN MOBILE — REGRA DE EXPERIÊNCIA

No mobile, o jogo deve buscar experiência de tela cheia sempre que o navegador ou PWA permitir. Implementar botão/ação “Entrar em Tela Cheia” usando Fullscreen API quando suportada, com fallback elegante quando indisponível.

Regras:

- fullscreen deve ser solicitado por interação explícita do usuário quando exigido pelo navegador;
- nunca travar o jogo se a API não estiver disponível;
- ocultar elementos de navegação internos não essenciais durante sessões ao vivo;
- respeitar notch, câmera, cantos arredondados e áreas seguras;
- manter botão claro para sair de fullscreen;
- recuperar layout corretamente ao alternar fullscreen, minimizar navegador ou girar dispositivo;
- em PWA instalada, aproveitar display standalone/fullscreen quando permitido.

A intenção é que no celular o jogo pareça um aplicativo nativo de gestão esportiva e não uma página web convencional.

165. ORIENTATION GUARD

Criar OrientationGuard global. Estados mínimos: LANDSCAPE_READY, PORTRAIT_BLOCKED, RESIZING e UNSUPPORTED_SIZE.

Em PORTRAIT_BLOCKED:

- pausar animações não essenciais;
- preservar sessão e save em memória;
- exibir arte de fundo premium;
- mostrar ícone de rotação;
- mensagem curta: “Gire o aparelho para jogar em tela horizontal.”

Ao retornar para landscape, restaurar exatamente o ponto anterior sem recarregar a carreira.

166. PC COMO EXPERIÊNCIA COMPLETA, NÃO PORT SECUNDÁRIO

No PC, o jogo deve usar o espaço adicional para ampliar informação e conforto, não para mudar regras. A mesma carreira, save, simulação e sistemas devem funcionar em mobile e desktop.

Desktop pode oferecer:

- sidebars persistentes;
- painéis simultâneos;
- tabelas com mais colunas;
- gráficos maiores;
- tooltips por hover;
- atalhos de teclado;
- drag quando útil;
- comparação lado a lado;
- mapa de pista maior;
- telemetria expandida.

Mobile usa painéis deslizáveis, abas contextuais, bottom sheets e drill-down. Desktop expande a mesma informação sem duplicar lógica.

167. DESIGN RESPONSIVO POR PRIORIDADE DE INFORMAÇÃO

Cada tela deve declarar três níveis de informação:

A — essencial: sempre visível;

B — contextual: acessível com um toque;

C — avançada: escondida em painel secundário, aba ou detalhes.

No mobile, nunca reduzir tudo até ficar ilegível. Deve haver reflow real. Exemplo: LiveTiming pode mostrar posição, nome curto, gap e pneu no modo compacto; detalhes de setores, stint e tendência aparecem ao tocar no piloto. No desktop, essas colunas podem permanecer abertas.

168. TAMANHOS-ALVO DE TELA E TESTES

O projeto deve ser testado em múltiplas proporções de tela horizontal, incluindo celulares compactos, celulares largos, tablets e desktop.

Faixas de validação recomendadas:

- 640×360 como piso funcional excepcional;
- 800×360/400 como mobile compacto;
- 844×390 e similares como referência moderna;
- 915×412 e similares como mobile amplo;
- 1024×600/768 como tablet/desktop compacto;
- 1366×768;
- 1920×1080;
- 2560×1440.

A UI deve continuar funcional mesmo fora dessas referências.

169. PERFORMANCE MOBILE COMO REQUISITO DE PARTE 1

A Parte 1 deve ser utilizável em smartphones intermediários atuais sem travamentos constantes. Priorizar:

- imagens WebP/AVIF quando possível;
- múltiplas resoluções por asset;
- lazy loading fora da sessão;
- pré-carregamento dos assets críticos da próxima tela;
- Canvas/SVG otimizado no TrackMap;
- atualização do LiveTiming por diff;
- evitar rerender completo por tick;
- evitar filtros CSS pesados contínuos;
- reduzir partículas em modo desempenho;
- suporte futuro a Web Worker para simulação.

Criar opções gráficas: Automático, Qualidade, Equilibrado e Desempenho.

170. PISTAS ORIGINAIS DO MUNDO TODO — REGRA INVIOLÁVEL

O calendário e a base de circuitos devem ser construídos para comportar pistas reais/originais de diferentes regiões do mundo. O jogador deve viajar por um campeonato verdadeiramente internacional, com circuitos visualmente e tecnicamente distintos.

A base deve contemplar progressivamente circuitos de:

- América do Sul;
- América do Norte;
- Europa;
- Ásia;
- Oceania;
- Oriente Médio;
- África quando houver pista adequada ao escopo da competição.

Não criar um campeonato onde todas as pistas parecem variações do mesmo traçado. Cada circuito precisa ter identidade geográfica, visual, climática e técnica.

Conteúdo real de nome, traçado e identidade oficial deve permanecer em camada de dados/assets licenciável e substituível, conforme as regras jurídicas já estabelecidas nesta Bíblia.

171. CIRCUITO COMO PERSONAGEM DO FIM DE SEMANA

Cada pista deve ser percebida como um “personagem” do calendário. O jogador precisa reconhecer rapidamente onde está antes mesmo de ler o nome.

Cada Circuit deve possuir:

- nome;
- país;
- cidade/região;
- bandeira do país;
- continente;
- timezone lógico do mundo do jogo;
- comprimento;

- curvas;
- sentido;
- altitude;
- velocidade média;
- perfil técnico;
- abrasividade;
- grip inicial;
- evolução de pista;
- probabilidade climática;
- temperatura típica;
- vento;
- risco de chuva;
- dificuldade de ultrapassagem;
- principais zonas de ataque;
- risco de queda;
- estresse de pneu dianteiro/traseiro;
- importância histórica/reputação;
- capacidade de público/relevância comercial quando aplicável.

172. BANDEIRAS DOS PAÍSES — ASSET OFICIAL

Toda pista deve exibir a bandeira correspondente ao país em pontos estratégicos da interface:

- calendário;
- card da etapa;
- briefing;
- loading da pista;
- classificação;
- grid;
- resultado;
- histórico;
- seleção de circuitos.

As bandeiras devem ser assets vetoriais ou raster de alta qualidade, com proporção correta e sem distorção. Não usar emoji como solução final de produção.

Criar assets/flags/<country-code>.svg ou equivalente e mapear por countryCode no asset-manifest.

173. FUNDO 4K EXCLUSIVO POR PISTA

Cada circuito principal deve possuir pelo menos um fundo 4K premium de estabelecimento visual, criado para representar aquele local de forma cinematográfica.

Famílias recomendadas por circuito:

- 1) heroBackground — imagem principal da etapa;
- 2) gridBackground — reta/grid antes da largada;
- 3) paddockBackground — contexto de paddock/garagens;
- 4) weatherBackground — variação climática quando viável;
- 5) resultsBackground — pós-corrida/pódio/arquibancada;

6) loadingBackground — carregamento da sessão.

Não é obrigatório que todas as pistas tenham seis imagens já na primeira versão, mas a arquitetura e o manifest devem suportar essas categorias desde a Parte 1.

174. DIREÇÃO DE ARTE DOS FUNDOS DE CIRCUITO

Fundos precisam ser lindos, realistas e imediatamente identificáveis. Diretrizes:

- 4K sempre que viável;
- composição cinematográfica horizontal 16:9 ou ultrawide adaptável;
- atmosfera condizente com país/região;
- arquitetura local quando aplicável;
- vegetação e relevo coerentes;
- arquibancadas, paddock e infraestrutura de corrida plausíveis;
- iluminação de manhã, tarde, pôr do sol ou clima compatível;
- áreas de respiro para UI;
- sem excesso de texto embutido na imagem;
- sem elementos que dificultem leitura dos painéis.

O Codex deve sobrepor UI sobre essas imagens usando gradientes, scrims e painéis com transparência controlada, nunca “queimar” texto diretamente no asset.

175. PACK VISUAL DA ETAPA

Ao entrar em uma etapa, a UI deve carregar um pacote visual do circuito:

- flagAsset;
- heroBackground;
- gridBackground;
- trackMapPath;
- miniTrackMap;
- countryName;
- circuitName;
- weatherProfile;
- atmosphereKey;
- accentTheme opcional;
- crowdProfile;
- ambientAudioKey futuro.

Isso permite que cada fim de semana pareça uma viagem diferente.

176. LOADING CINEMATOGRAFICO DE CIRCUITO

A transição para uma etapa deve apresentar:

- fundo 4K da pista;
- bandeira do país;
- nome da pista;
- cidade/região;
- rodada da temporada;
- extensão;
- número de curvas;
- clima inicial;

- recordes internos do save quando existirem;
- pequena dica técnica do engenheiro;
- progresso de carregamento real.

Nunca usar loading falso longo. Se o carregamento terminou, liberar a etapa.

177. MAPA 2D DAS PISTAS — FIDELIDADE E LEGIBILIDADE

O TrackMap deve usar o desenho real/original do layout disponibilizado na camada de circuito, respeitando formato, direção e posição relativa das principais curvas.

Mesmo sendo 2D, deve transmitir qualidade premium:

- traçado limpo;
- linha de pista com espessura responsiva;
- setores visualmente marcados;
- pit lane identificável;
- linha de largada/chegada;
- motos/ícones legíveis;
- destaque da equipe do usuário;
- bandeiras por setor;
- chuva ou condição de pista por trechos quando suportado;
- zoom automático opcional para batalhas.

178. GRID VISUAL ESPECÍFICO DE CADA PISTA

O grid ao vivo deve usar o gridBackground daquele circuito. A UI deve combinar:

- reta principal;
- bandeira do país;
- identificação da pista;
- clima;
- temperatura;
- slots dos pilotos;
- retratos fictícios;
- números;
- equipes;
- fabricantes;
- penalidades;
- animação de entrada.

O fundo nunca deve ser uma imagem genérica igual em todas as etapas.

179. CALENDÁRIO INTERNACIONAL E SENSACÃO DE VIAGEM

O calendário deve mostrar progressão geográfica e sensação de mundo. Cada rodada possui card com bandeira, país, pista, data do mundo do jogo, distância logística aproximada, clima esperado e importância comercial.

A transição entre continentes deve afetar logística, fadiga de staff, custos, prazo de peças e preparação.

O usuário deve sentir que sua equipe está viajando pelo planeta e não apenas trocando uma variável de circuito.

180. PERFIS CLIMÁTICOS POR CIRCUITO

Cada pista deve ter WeatherProfile próprio com distribuição sazonal e probabilidade contextualizada de:

- seco estável;
- nublado;
- chuva leve;
- chuva forte;
- pista secando;
- calor extremo;
- frio;
- vento elevado;
- mudanças rápidas.

O clima não precisa replicar meteorologia real em tempo real. Deve ser plausível para o local e temporada do jogo.

181. PROGRESSÃO DE CARREIRA ENTRE CATEGORIAS

A carreira da equipe pode começar em nível inferior e ascender. A promoção não deve ser automática por botão. Depende de reputação, resultados, orçamento, infraestrutura, convite, fabricante, pilotos e critérios da categoria.

Possibilidades:

- permanecer e dominar categoria atual;
- subir de categoria;
- abrir programa paralelo de base;
- aceitar convite de campeonato superior;
- comprar vaga quando regras fictícias permitirem;
- recusar subida por risco financeiro;
- retornar a categoria menor em crise.

A progressão deve criar uma história de organização, não apenas troca de estatísticas.

182. MATRIZ DE DEPENDÊNCIA ENTRE SISTEMAS — REGRA OBRIGATÓRIA

Nenhum sistema importante pode operar isoladamente. Criar dependencyGraph conceitual e testes de integração.

Exemplos obrigatórios:

- lesão do piloto → disponibilidade → substituto → contrato → performance → moral → mídia → patrocinadores → finanças;
- queda forte → dano de moto → estoque → mecânicos → custo → confiabilidade → sessão seguinte;
- vitória inesperada → reputação → fãs → patrocinadores → salário esperado → interesse de rivais;
- troca de fabricante → adaptação técnica → staff → setup → estoque → marketing → orçamento;

- crise financeira → P&D → qualidade de peças → resultados → moral → retenção de talentos;
- campeonato internacional → logística → fadiga → risco operacional → custos.

Quando um evento relevante acontecer, o EventBus deve propagar consequências coerentes aos sistemas relacionados.

183. LESÕES, CONDIÇÃO FÍSICA E RECUPERAÇÃO

Pilotos devem possuir fitnessState: condição, fadiga, dor, recuperação, risco de agravamento e clearance médico.

Lesões podem ser leves, moderadas ou graves, sem exploração gráfica. Impactos possíveis:

- perda de performance;
- limitação de treino;
- ausência de sessão;
- ausência de etapa;
- redução temporária de agressividade/confiança;
- necessidade de substituto;
- custo médico/fisioterapia;
- impacto de mídia e patrocinadores.

Equipe médica e fisioterapia melhoram diagnóstico e recuperação, mas não eliminam risco.

184. MERCADO DE MEIO DE TEMPORADA, RUMORES E TRANSFERÊNCIAS

O mercado deve permanecer ativo durante a temporada. Criar:

- rumores;
- sondagens;
- pré-contratos quando permitidos pelo universo do jogo;
- interesse confidencial;
- cláusulas de saída;
- buyout;
- disputas entre equipes;
- renovação antecipada;
- substituição por desempenho;
- pilotos temporários.

Notícia pública não pode revelar automaticamente informação privada verdadeira. Rumores podem ter nível de credibilidade.

185. ACADEMIA E CATEGORIAS DE BASE

Equipes podem criar programa de jovens. O sistema inclui:

- orçamento de academia;
- scouts regionais;
- pilotos vinculados;
- empréstimo/colocação em equipes menores;
- bolsas e suporte;
- progresso;

- direito preferencial de contrato;
- risco de perder talento para rival.

Isso alimenta a sucessão de pilotos em carreiras longas.

186. LEGADO, RECORDES E HISTÓRIA

O jogo deve construir um arquivo histórico vivo:

- campeões por temporada;
- títulos de equipe;
- vitórias;
- poles;
- pódios;
- recordes de pista internos do save;
- maiores sequências;
- maiores recuperações;
- estreias;
- aposentadorias;
- transferências marcantes;
- rivalidades;
- equipes extintas ou renomeadas quando aplicável.

O objetivo final da carreira não é apenas dinheiro ou troféus; é construir legado reconhecível dentro do universo.

187. BÍBLIA VISUAL — ROADMAP DE ASSETS

Criar lista numerada de assets a serem produzidos aqui no chat. Regra-mãe: cada item numerado corresponde a uma imagem individual, nunca colagem.

Categorias prioritárias:

- logo oficial;
- capa;
- splash;
- menu;
- dashboard background;
- sede;
- garagem;
- oficina;
- engenharia;
- pit wall;
- paddock;
- marketing;
- sala de contratos;
- scouting;
- retratos de pilotos;
- retratos de staff;
- fundos de pistas;
- grid backgrounds;
- podium backgrounds;

- loading de pistas;
- UI frames;
- cards;
- badges;
- ícones de moto;
- weather visuals;
- alertas;
- celebrações.

Produção deve seguir prioridade de implementação do Codex para que assets essenciais existam antes das telas que dependem deles.

188. QA MOBILE E DESKTOP — CHECKLIST DE ACEITAÇÃO

Nenhuma tela é considerada pronta sem testar:

- landscape mobile;
- fullscreen/fallback;
- notch/safe area;
- touch sem hover;
- texto legível;
- botões acessíveis com polegar;
- ausência de scroll horizontal acidental;
- resize e rotação;
- retorno de background/foreground;
- desktop com mouse;
- teclado quando aplicável;
- 1366×768;
- 1920×1080;
- tempo de carregamento;
- estabilidade do TrackMap;
- LiveTiming em sessão longa.

189. REQUISITOS ADICIONAIS DA PARTE 1

A Parte 1 passa a exigir também:

- layout mobile-first horizontal;
- OrientationGuard;
- experiência fullscreen quando suportada;
- PC responsivo completo;
- pelo menos um conjunto inicial robusto de pistas internacionais;
- bandeira por circuito;
- fundo premium por pista usada na temporada inicial;
- TrackMap próprio por circuito;
- loading de etapa;
- grid usando fundo do circuito;
- clima por perfil de pista;
- performance configurável para mobile;
- testes de layout e sessão em smartphones.

Esses itens não devem ser adiados integralmente para Parte 2 ou 3.

190. DEFINITION OF DONE — PLATAFORMA, PISTAS E EXPERIÊNCIA GLOBAL

Este bloco só estará concluído quando o jogo demonstrar uma carreira jogável em mobile horizontal, com fullscreen quando suportado, PC responsivo, múltiplas pistas internacionais distintas, bandeiras corretas, fundos 4K premium, TrackMaps próprios, loading e grid contextualizados, diferenças técnicas entre circuitos, clima plausível por local e ausência de telas quebradas em proporções móveis comuns.

A experiência deve transmitir uma frase simples: “Estou administrando uma equipe mundial de motovelocidade e cada etapa acontece em um lugar realmente diferente.”

191. LOOP DE GAMEPLAY MINUTO A MINUTO

O jogo precisa definir com clareza o que o jogador faz em cada escala de tempo. A experiência deve alternar entre decisões rápidas, planejamento de curto prazo e estratégia de longo prazo.

Escala imediata: abrir alerta, responder proposta, aprovar tarefa de garagem, alterar setup, chamar piloto aos boxes, reagir a clima, ajustar risco ou ritmo.

Escala diária/semanal: analisar finanças, mercado, staff, P&D, logística, patrocinadores, mídia e preparação para a próxima etapa.

Escala de temporada: objetivos, evolução da equipe, renovação de contratos, mudança de fabricante, academia, infraestrutura, expansão de categoria e legado.

O jogador nunca deve ficar sem um objetivo compreensível, mas também não deve ser bombardeado por microtarefas irrelevantes.

192. ECONOMIA PROFUNDA E SOBREVIVÊNCIA DA EQUIPE

A economia deve permitir sucesso, estagnação, reconstrução e crise real. O jogo precisa acompanhar caixa atual, receitas previstas, despesas contratadas, compromissos futuros, dívida, reservas de emergência, risco de insolvência e valor estimado da equipe.

Criar níveis de saúde financeira: saudável, atenção, pressionada, crítica e insolvência. Cada faixa deve alterar comportamento de investidores, fornecedores, pilotos, staff e patrocinadores.

Uma equipe pode escolher cortar custos, congelar P&D, vender ativos, renegociar contratos, buscar aporte, assumir dívida, reduzir hospitalidade ou apostar em resultados esportivos para sobreviver.

193. VALOR DA EQUIPE, INVESTIDORES E PARTICIPAÇÃO

Criar TeamValuation baseado em marca, resultados, ativos, contratos, base de fãs, infraestrutura, reputação, receitas recorrentes e risco financeiro.

Investidores podem oferecer capital em troca de participação, influência ou metas. O jogador deve poder aceitar, rejeitar ou negociar. Vender participação melhora caixa, mas pode reduzir autonomia e aumentar pressão por resultados.

194. STAFF — MORAL, LEALDADE, FADIGA E CONFLITO

Staff precisa ser tratado como grupo humano vivo. Cada profissional possui moral, lealdade, fadiga, satisfação salarial, satisfação com função, relação com liderança e intenção de permanência.

Eventos possíveis: pedido de aumento, assédio de rival, queda de motivação, conflito entre departamentos, elogio por resultado, estafa de calendário, promoção interna e ameaça de saída.

Fadiga deve aumentar em viagens, sequências de etapas e crises, impactando velocidade de trabalho e risco de erro.

195. EVOLUÇÃO E CARREIRA DO STAFF

Profissionais devem desenvolver especialidades, reputação e valor de mercado. Engenheiros podem virar chefes técnicos; mecânicos podem se tornar crew chiefs; scouts podem virar diretores de academia.

Staff também envelhece, muda de prioridade e pode se aposentar. O mundo precisa produzir novos profissionais para carreiras longas.

196. PILOTOS — CONFIANÇA, PRESSÃO E ESTADO EMOCIONAL

Separar atributos permanentes de estados temporários. Criar confidence, pressure, frustration, motivation e teamTrust.

Vitórias, erros, críticas, contratos, ordens de equipe, rivalidades, lesões e desempenho do companheiro alteram esses estados.

Estados temporários não devem gerar bônus absurdos, mas precisam influenciar consistência, risco, tomada de decisão e comunicação.

197. RIVALIDADES ENTRE PILOTOS E EQUIPES

Rivalidades devem nascer de eventos reais do save: contatos, disputas por título, declarações, contratos, ultrapassagens agressivas, ordens de equipe e histórico de resultados.

Cada rivalidade possui intensidade, memória e motivo. Rivalidade pode aumentar mídia, fãs e pressão, mas também gerar risco esportivo e desgaste interno.

198. IA RIVAL — MODELO DE PERSONALIDADE DE EQUIPE

Cada equipe IA deve ter identidade estratégica: conservadora, agressiva, formadora, técnica, comercial, fabricante, oportunista ou premium.

A IA decide orçamento, mercado, P&D, pilotos, staff e estratégia respeitando filosofia, recursos e contexto. Ela deve errar de forma plausível e aprender com resultados ao longo das temporadas.

199. EVENTOS DINÂMICOS DO MUNDO

Criar EventGenerator conectado ao estado real do jogo. Tipos possíveis: lesão, quebra crítica, punição, mudança de regulamento, crise de patrocinador, convite de categoria, protesto técnico, atraso logístico, mudança de clima extremo, saída de staff, rumor de mercado, acidente em treino, oportunidade comercial, bônus de fabricante e crise financeira.

Eventos nunca devem ser aleatórios sem contexto. Cada evento deve possuir causa provável, impacto, duração, opções de resposta e sistemas afetados.

200. REGULAMENTO DE CORRIDA — BANDEIRAS E INTERRUPÇÕES

A engine deve suportar bandeira verde, amarela local, amarela geral quando aplicável, vermelha, quadriculada e condições especiais definidas pelo campeonato.

Bandeira vermelha deve interromper sessão e aplicar regras de restart configuráveis. Incidentes devem poder gerar neutralização, pausa, reinício, redução de distância ou encerramento conforme regulations data.

201. PARC FERMÉ, RESTARTS E DECISÕES ESPORTIVAS

Criar regras configuráveis de parc fermé, mudanças permitidas após classificação, alterações por condição climática, formação de novo grid após interrupção e classificação em corrida interrompida.

Toda decisão deve vir do rulesEngine e gerar explicação visível ao jogador.

202. PWA E INSTALAÇÃO NO CELULAR

O jogo deve ser preparado como Progressive Web App quando viável: manifest, ícone, nome curto, tela de abertura, instalação na tela inicial, display standalone/fullscreen quando suportado e service worker.

A experiência instalada deve abrir diretamente no jogo, manter orientação horizontal e parecer próxima de aplicativo nativo.

203. CACHE, OFFLINE E CARREGAMENTO INTELIGENTE

Assets estáticos essenciais, UI base e dados já carregados podem usar cache controlado. Nunca armazenar estado crítico apenas no cache.

Se conexão cair durante jogo local, a carreira deve continuar quando possível. Funções externas ou futuras online devem falhar de forma elegante.

204. AUTOSAVE E MÚLTIPLOS SLOTS

Criar autosave seguro em momentos definidos: antes de sessão, após sessão, após corrida, após contratação relevante e ao avançar calendário.

Suportar múltiplos slots de carreira, slot manual e backup do último save válido. Nunca sobrescrever o único estado válido sem cópia de segurança quando houver migração ou operação crítica.

205. RECUPERAÇÃO DE SAVE CORROMPIDO

Se um save falhar ao carregar, tentar backup anterior, migração ou validação parcial. O jogo deve informar claramente o problema e nunca apagar automaticamente toda a carreira sem tentativa de recuperação.

206. MONETIZAÇÃO E EXPANSÕES — PRINCÍPIO ÉTICO

O design principal não deve depender de pay-to-win, energia artificial, loot boxes ou bloqueio abusivo de progressão.

Se houver monetização futura, priorizar compra premium, expansões de conteúdo, pacotes de pistas/categorias licenciadas, edições especiais ou conteúdo cosmético, sem vender vantagem escondida dentro da simulação.

207. BÍBLIA VISUAL DEFINITIVA — DESIGN SYSTEM

Criar especificação visual final com: tipografia, escala, espaçamento, grid, radius, sombras, bordas, opacidades, estados de botão, cards, modais, tabelas, inputs, sliders, badges, tooltips, overlays, scrims e linguagem de animação.

Toda tela deve parecer parte do mesmo produto. O Codex não pode inventar um estilo diferente em cada módulo.

208. HUD E HIERARQUIA VISUAL EM SESSÕES

No mobile horizontal, o HUD de sessão deve priorizar TrackMap, posição, gaps, volta, clima, pneus, status da moto e comandos críticos.

Informações secundárias entram por toque, drawer ou bottom sheet. No PC, podem permanecer simultaneamente visíveis em painéis adicionais.

Nunca cobrir o mapa com excesso de janelas. O jogador precisa acompanhar corrida e decidir sem navegar por menus profundos.

209. ACESSIBILIDADE AVANÇADA

Além das regras já definidas, incluir: escala de UI, tamanho de texto, modo alto contraste, redução de movimento, vibração opcional, sons de alerta configuráveis, símbolos adicionais além de cor, confirmação de ações críticas e modo de leitura simplificada.

210. TUTORIAL REVISITÁVEL E CENTRAL DE AJUDA

Após o tutorial inicial, o usuário deve poder reabrir explicações por sistema. Criar HelpCenter contextual com artigos curtos, glossário, exemplos e botão “mostrar na tela”.

211. CHECKLIST DE QA POR TELA

Cada tela deve possuir checklist: abre sem erro, dados corretos, estado vazio, estado carregando, estado erro, toque, mouse, teclado quando aplicável, mobile landscape, desktop, fullscreen, rotação, retorno de background e persistência após reload.

212. TESTES DE INTEGRAÇÃO ENTRE SISTEMAS

Além de testes unitários, criar cenários completos: piloto lesionado gera substituição e impacto financeiro; patrocinador perdido altera projeção; mudança de fabricante afeta moto e contratos; vitória aumenta reputação e mercado; crise financeira reduz P&D e competitividade.

213. TELEMETRIA DE QUALIDADE DO PRODUTO EM MODO DEV

Modo desenvolvedor deve registrar tempo de carregamento, duração de frames lentos, número de entidades ativas, tamanho de save, erros de UI, eventos não tratados e módulos mais pesados.

Essas métricas servem para otimização, nunca para enviar dados pessoais sem consentimento.

214. PLANO DE PRODUÇÃO DO CODEX — PARTE 1 REFINADA

Parte 1 deve entregar uma carreira completa e estável com: plataforma mobile-first horizontal; PWA base; criação de equipe; elenco fictício; staff; contratos; economia; patrocinadores; calendário internacional; pistas com bandeiras, TrackMaps e fundos premium; treino; classificação; grid; corrida 2D; clima; pneus; setup; IA; save; autosave; resultados; temporada completa; tutorial; dashboard; QA e testes.

A Parte 1 não termina enquanto o usuário não puder iniciar, jogar e concluir uma temporada inteira em mobile e PC sem depender de funções falsas.

215. PLANO DE PRODUÇÃO DO CODEX — PARTE 2 REFINADA

Parte 2 aprofunda tudo que já funciona: P&D avançado, staff profundo, mercado dinâmico, academia, lesões, rivalidades, eventos, clima avançado, regulamento complexo, IA estratégica, mídia, investidores, logística, histórico e mais assets 4K.

Não substituir sistemas da Parte 1. Apenas aprofundar e ampliar.

216. PLANO DE PRODUÇÃO DO CODEX — PARTE 3 REFINADA

Parte 3 concentra lapidação AAA: animação, áudio, cinematografia, performance mobile, acessibilidade completa, otimização, balanceamento em larga escala, UX final, expansões, editor de dados quando viável, documentação final e preparação de release.

217. GATE DE ACEITAÇÃO ENTRE PARTES

Codex não avança de parte se a anterior tiver funções essenciais quebradas. Cada etapa exige checklist assinado pelo próprio processo de desenvolvimento: testes passando, save estável, mobile jogável, desktop jogável, sem botões falsos e sem placeholders críticos.

218. PROMPT MESTRE DO CODEX — REGRA DE USO

Antes de iniciar qualquer uma das três partes, o Codex deve receber um prompt mestre que instrua: ler toda esta Bíblia; respeitar prioridades; não simplificar sistemas essenciais; preservar compatibilidade mobile/PC; usar assets oficiais; implementar dados centralizados; testar; documentar; e não declarar concluído o que não funciona.

O prompt mestre deve citar explicitamente os critérios de aceite da parte correspondente.

219. AUDITORIA FINAL DA BÍBLIA

Antes de congelar a versão de produção, revisar se existem lacunas em: carreira, economia, staff, pilotos, IA, corrida, pistas, mobile, PC, PWA, save, assets, acessibilidade, QA, monetização, expansões e pipeline do Codex.

Qualquer sistema citado como essencial deve possuir: objetivo, dados, lógica, UI, persistência, consequência e teste.

220. DEFINITION OF DONE — BÍBLIA MESTRA PRONTA PARA PRODUÇÃO

A Bíblia estará pronta para orientar o Codex quando: todas as regras centrais estiverem documentadas; os sistemas essenciais estiverem conectados; a experiência mobile horizontal estiver definida; PC estiver coberto; pistas internacionais tiverem pipeline visual e técnico; pilotos e staff fictícios estiverem especificados; a estrutura em três partes estiver fechada; e existir um caminho claro de implementação, teste, balanceamento e release.

A regra final do projeto permanece: não criar apenas um jogo funcional. Criar um manager de motovelocidade profundo, visualmente premium, tecnicamente organizado, jogável no celular e no PC, com identidade própria e capacidade de sustentar muitas temporadas sem perder imersão.